

ORCA: ORchestrating Causal Agent

Joanie Hayoun Chung
Department of Statistics, Korea
University
Seoul, South Korea
jchung02@korea.ac.kr

Sumin Lee
Department of Statistics, Korea
University
Seoul, South Korea
leesumin0924@korea.ac.kr

Sunbin Lim
Department of Statistics, Korea
University
Seoul, South Korea
LG AI Research
Seoul, South Korea
sunbin@korea.ac.kr

Abstract

Causal analysis on relational databases is challenging, as analysis datasets must be repeatedly queried from complex schemas. Recent LLM systems can automate individual steps, but they hardly manage dependencies across analysis stages, making it difficult to preserve consistency between causal hypothesis. We propose ORCA (ORchestrating Causal Agent), an interactive multi-agent framework to enable coherent causal analysis on relational databases by maintaining shared state and introducing human checkpoints. In a controlled user study, participants using ORCA successfully completed end-to-end analysis more often than with a baseline LLM (GPT-4o-mini) assistant by 42 percentage points, achieved substantially lower ATE error, and reduced time spent on repetitive data exploration and query refinement by 76% on average. These results show that ORCA improves both how users interact with the causal analysis pipeline and the reliability of the resulting causal conclusions.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; *Causal reasoning and diagnostics*; • **Information systems** → *Data analytics*.

Keywords

Causal inference, Causal discovery, Causal agent, Interactive data analysis

ACM Reference Format:

Joanie Hayoun Chung, Sumin Lee, and Sunbin Lim. 2026. ORCA: ORchestrating Causal Agent. In *Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems (CHI EA '26)*, April 13–17, 2026, Barcelona, Spain. ACM, New York, NY, USA, 35 pages. <https://doi.org/10.1145/3772363.3798444>

Corresponding author. E-mail: sunbin@korea.ac.kr.

1 Introduction

Understanding causality is essential in high-stakes domains such as medicine, public policy, and business strategy, where analytical errors can directly affect safety, equity, and resource allocation [4, 10, 14]. Reliable causal analysis requires both domain knowledge to articulate a plausible data-generating process and statistical expertise to select valid identification and estimation strategies. These requirements are intensified by large, relationally complex databases, where variables are distributed across multiple tables [3, 6, 21]. Analysts must obtain datasets through repeated joins, aggregations, and variable definitions. This iterative process is often time-consuming, and because these decisions determine the inputs to causal models, even small ambiguities in early dataset exploration can propagate across stages and distort downstream causal conclusions. Recent LLM systems can automate parts of this workflow [15, 28, 29], particularly SQL generation and code execution [32]. Yet they often fail to manage dependencies between data-related and modeling decisions across stages, leaving analysts without support for preserving consistency across the workflow.

Under these conditions, we propose ORCA (ORchestrating Causal Agent), an interactive multi-agent framework for reliable causal analysis on relational databases. ORCA decomposes the workflow into specialized agents for data exploration, causal discovery, and effect estimation, coordinated by an Orchestrator that maintains cross-stage consistency and mediates user feedback. Rather than fully automating decisions, ORCA introduces checkpoints that allow analysts to approve or revise artifacts. The main contributions of this paper are as follows: (1) We introduce ORCA, an interactive multi-agent system that supports the workflow of causal analysis on relational databases by guiding user interaction through explicit checkpoints. (2) We demonstrate that ORCA improves causal outcome quality by enforcing cross-stage consistency, yielding more accurate graphs and ATE estimates. (3) We present REEF, a semi-synthetic relational e-commerce benchmark with an explicit data-generating process, and evaluate ORCA via controlled experiments and a user study showing higher completion rates, lower ATE error, and reduced exploration time.

2 ORCA: Orchestrating Causal Agent

2.1 Research Motivation and Framework Overview

To investigate practical challenges in LLM-assisted causal analysis, we conducted a user study with twelve participants performing end-to-end causal tasks on a relational database using a GPT-4o-mini



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

CHI EA '26, Barcelona, Spain

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2281-3/2026/04

<https://doi.org/10.1145/3772363.3798444>

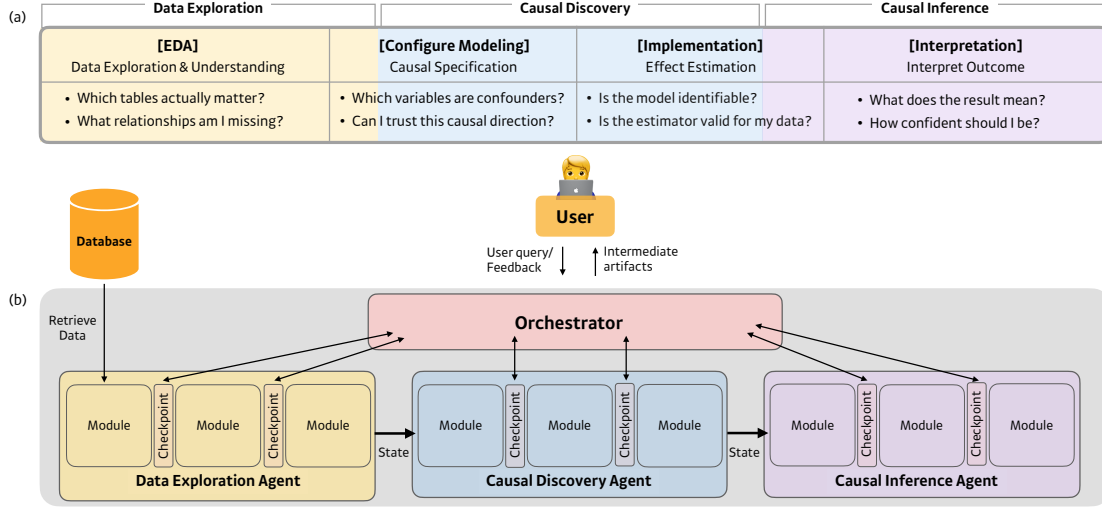


Figure 1: (a) The step-wise causal analysis procedure, highlighting the analytical questions that arise at each stage. (b) ORCA is aligned with this procedure by an orchestrated multi-agent framework. Three specialized agents share state of resulting artifacts from each module, while a central Orchestrator operates these agents sequentially during the analysis. User interaction is mediated by the Orchestrator through checkpoints where intermediate artifacts are exposed for inspection and feedback.

assistant with code execution tools. Even with automated assistance, participants spent 77% of their total analysis time on data exploration alone (see Figure 5(a)). Repeated exploration made it difficult to track whether earlier intermediate artifacts remained valid, leading many participants to proceed without explicitly verifying these outputs. This is critical in causal discovery and inference, since these methods are only applicable when the assumptions they rely on hold. As a result, causal estimation was frequently applied to datasets that no longer satisfied the assumptions required for valid inference, yielding poor estimation outcomes. Only 8.3% of final ATE estimates fell within the ground-truth confidence interval (see Table 1). See Section 3.2 for detailed quantitative and qualitative results. These observations suggest that the core challenge lies not in automating individual steps, but in maintaining coherence across interdependent stages of causal analysis. We formalize this as cross-stage consistency: the set of constraints requiring that (i) the relational join and aggregation decisions made during data exploration are reflected in the variable definitions passed to causal discovery, (ii) the causal graph produced by discovery encodes only assumptions that hold over the constructed dataset, and (iii) the identification strategy selected for effect estimation remains valid under those same assumptions. Violations at any stage, such as changing a variable’s aggregation level after a causal graph has been constructed can silently invalidate downstream estimates.

ORCA is designed to address this problem by structuring causal analysis as a sequence of stages with explicit artifacts and checkpoints. It automates routine operations while preserving shared state and enabling users to inspect and revise intermediate results before errors propagate. Figure 1 illustrates this stage-wise view and shows how ORCA realizes it through an orchestrated agent framework.

The following subsections describe each specialist agent in terms of its responsibilities and checkpoints within the end-to-end causal workflow. Implementation details are provided in Appendix B for reproducibility.

2.2 Data Exploration Agent

The Data Exploration Agent supports preparation of analysis-ready datasets. In relational databases, data exploration is a major bottleneck, involving repeated table lookup, interpretation of unfamiliar columns, and data quality assessment. The agent decomposes this process into modules that leverage precomputed metadata and column-level statistics to guide exploration, producing artifacts that are validated at checkpoints and stored as shared state for consistency.

As illustrated in Figure 2, the Data Exploration Agent consists of four modules that can be used independently or sequentially. (1) The *Table Explorer* supports early-stage sense-making by summarizing table semantics, data quality, and relational context. By integrating column-level statistics with schema metadata, it helps users assess whether a table is suitable for defining treatment or outcome variables. (2) The *Table Recommender* determines the relational scope of the analysis by identifying relevant tables and valid join paths aligned with the user’s analytical intent. (3) The *Text2SQL Generator* translates natural language requests into SQL queries and iteratively validates them for both syntactic and semantic alignment with the intended unit of analysis. (4) Finally, the *Data Preprocessor* applies lightweight transformations and annotates variable types and cardinalities, producing an analysis-ready dataset.

The effectiveness of these modules is evaluated on the REEF and BIRD benchmarks, demonstrating higher-quality table descriptions, more accurate table retrieval, and more reliable Text2SQL

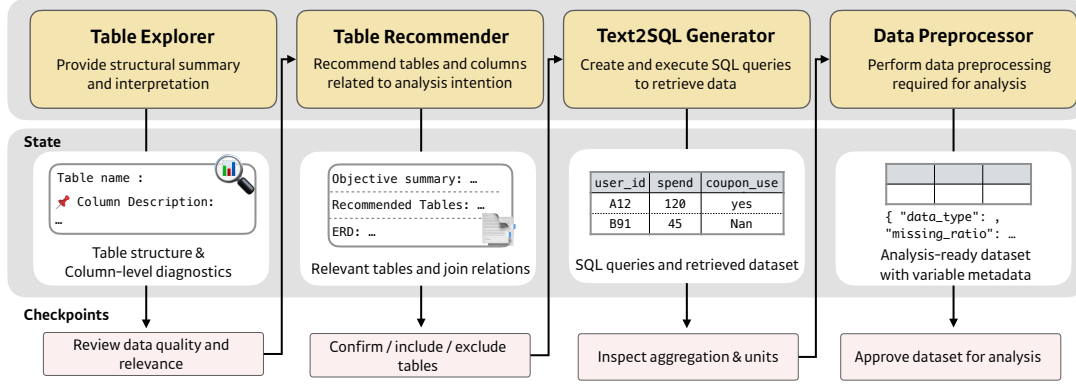


Figure 2: ORCA’s Data Exploration Agent and its interaction structure. The agent prepares an analysis-ready dataset through four modules. Each module produces an intermediate artifact saved in state (middle row) exposing table semantics, relational scope, query structure, and outcomes. At each checkpoint (bottom row), users review and validate these outputs before proceeding.

execution; detailed evaluation metrics and results are provided in Appendix D.2. Implementation-level details of the Data Exploration Agent, including module-specific prompts, inputs and outputs, and execution logic, are provided in Appendix B.1.

2.3 Causal Discovery Agent

The Causal Discovery Agent aims to construct a reliable causal graph. Causal discovery is inherently ill-posed [24], as multiple graph structures can be consistent with the same observational distribution, and no single modeling assumption can be fixed a priori [27, 34]. Recent LLM-based approaches assist discovery by proposing structural hypotheses or incorporating human-provided cues [13], but often rely on implicit assumptions not validated against data, leading to fragile or misleading graphs [12]. To address this, ORCA adopts a diagnostics-driven approach that evaluates assumption-relevant data properties and guides users to interpret and inspect the decisions and artifacts.

As illustrated in Figure 3, the Causal Discovery Agent consists of four sequential modules with shared state. (1) The *Testing Causal Hypothesis* module evaluates whether key assumptions such as linearity and noise properties are supported by the data, constraining the applicable algorithm classes. (2) The *Algorithm Configuration* module maps these diagnostics to compatible algorithm families and requests user approval. (3) The *Graph Generation and Scoring* module generates candidate causal graphs using the configured algorithms and compares them using stability and consistency criteria, enabling user inspection of alternative structures. (4) Finally, the *Ensemble and Synthesis* module aggregates evidence across candidates to output both an uncertainty-preserving graph for inspection and a fully directed acyclic graph for inference. Users review the synthesized graph before used for causal estimation.

Across synthetic settings, the agent yields more stable causal graphs, consistently reducing structural error (SHD) in scale-free settings (See Appendix D.3 for full experimental results). Appendix B.2 specifies the procedures, diagnostic tests, thresholds, scoring definitions, and synthesis algorithms used.

2.4 Causal Inference Agent

The Causal Inference Agent supports reliable effect estimation through step-wise inference with checkpoints and dependency-aware shared state. Given both a dataset and causal graph, the agent ensures consistency in variable definition, identification, and estimation strategies while explicitly reporting when analyses are not applicable.

As illustrated in Figure 4, the Causal Inference Agent is organized into four modules. (1) The *Variable Selection* module grounds the user’s causal query by fixing the treatment and outcome with respect to the available data and assigns remaining variables to causal roles based on the selected graph, which users validate against the intended causal question. (2) The *Configuration Selection* module determines an appropriate identification and estimation strategy by matching data characteristics and graph structure to compatible causal assumptions. (3) The *Model Implementation* module executes effect estimation when the chosen configuration is identifiable, and explicitly reports failure when structural or statistical requirements are not met, avoiding spurious estimates. (4) Finally, the *Interpretation* module summarizes the estimated effect, identification conditions, and limitations that govern how the result should be interpreted.

ORCA enables robust causal inferences achieving consistently low absolute error (as low as 3.2×10^{-5} on REEF) and maintaining high confidence interval coverage (up to 100%). See Appendix B.3 and Appendix D.4 for implementation details and evaluation results.

3 User Study & Design Implications

3.1 Study Design

We conducted a user study to evaluate ORCA using a within-subjects design with twelve data-literate participants who regularly work with Python and SQL, are familiar with causal concepts, but have comparatively less hands-on experience applying causal inference methods. Each participant completed two causal analysis tasks on the same relational database—one with ORCA and one with a baseline assistant (GPT-4o-mini) featuring tool access, step-gated progression, and minimal state persistence—in counterbalanced

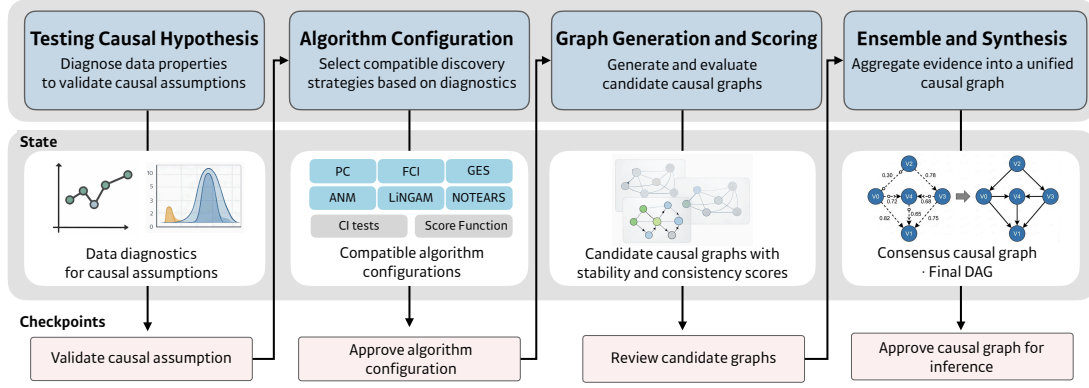


Figure 3: ORCA's diagnostics-driven Causal Discovery Agent with user checkpoints. Starting from a preprocessed dataset, the agent evaluates causal assumptions, configures compatible discovery strategies, and generates candidate causal graphs. Intermediate artifacts in state (middle row) expose diagnostics, configurations, and candidate graphs. Checkpoints (bottom row) prompt users to validate assumptions, approve configurations, and review graph structures before inference.

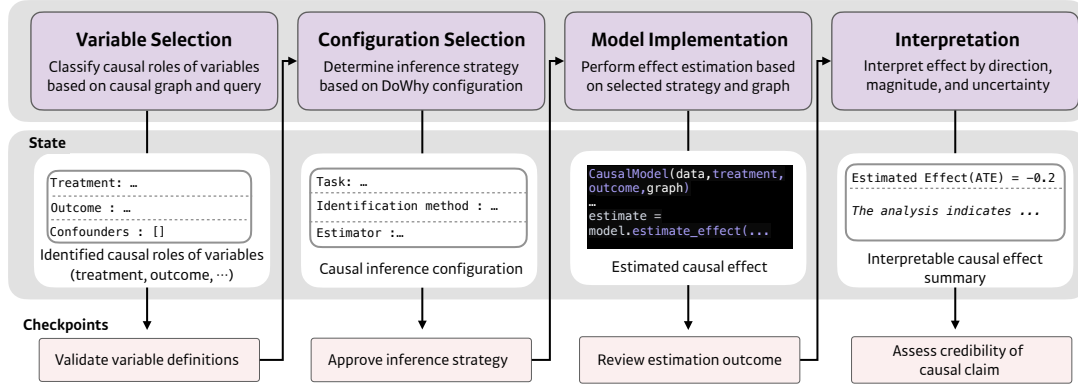


Figure 4: ORCA's Causal Inference Agent. Given a causal graph and an analysis-ready dataset, the agent selects inference strategy, executes effect estimation, and interprets results. Intermediate artifacts in state (middle row) expose variable roles, inference strategy, and estimation outputs and at each checkpoint (bottom row), users validate decisions and assess the results.

order. Each task, framed as a standardized average treatment effect (ATE) query, followed a three-stage pipeline of determining the analysis dataset, specifying the causal graph, and estimating the causal effect, and was limited to 60 minutes. We recorded interaction logs, task outcomes, and pre- and post-task survey responses. Detailed demographics and the study protocol are provided in Appendix E.

To our knowledge, no existing benchmark provides a relational database with a known ground-truth causal graph and effect estimates, both of which are required for controlled evaluation of end-to-end causal analysis workflows. For evaluation, we construct REEF (Relational E-commerce Evaluation Framework), a semi-synthetic relational database reflecting real-world challenges of causal analysis on relational data. In REEF, variables relevant to a causal query are distributed across multiple tables, requiring users to determine join paths, aggregation levels, and units of analysis. The database is generated from a predefined Structural Causal Model (SCM) that specifies causal dependencies, enabling evaluation against a known causal structure. REEF also incorporates

realistic data characteristics such as missing values and inactive entities. See Appendix C for SCM specifications and data generation procedures.

3.2 Results and Implications

Findings 1: Improved causal outcome quality through reduced upstream errors. As shown in Figure 5(c), ORCA produces ATE estimates more concentrated near zero, with a smaller mean absolute error (0.003 vs. 0.06). Even when all trials are included, the gap becomes substantially larger (0.06 vs. 18.07), indicating that baseline errors are not only more dispersed but also prone to extreme failures. Such failures were largely driven by cases where estimation was applied to mis-specified datasets or under unverified assumptions that violated identification conditions, a failure mode ORCA is specifically designed to prevent. Consistent with this, ORCA produces causal graphs with 31% lower Normalized Hamming Distance (see Figure 5(b)), indicating that structural improvements earlier in the pipeline carry forward into more reliable estimates. As shown

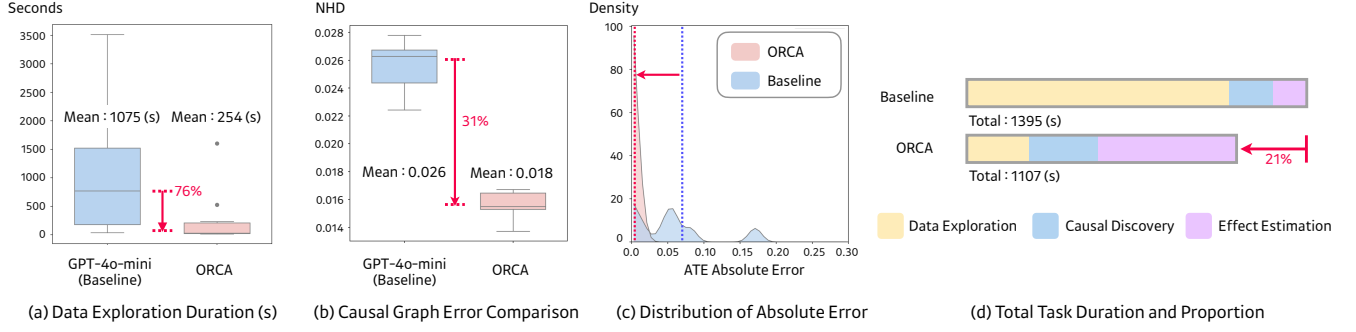


Figure 5: (a) Boxplot of time spent at data exploration analysis step. ORCA significantly reduces time spent on early-stage data exploration. (b) Comparison on structural error of the inferred causal graph. ORCA consistently produces more accurate graphs than the baseline. (c) Distribution of participants reports over Average Treatment Effect (ATE) absolute error. Outliers are omitted for readability. (d) Total task duration and proportion of total task time spent at each step. ORCA shifts user effort towards human-central causal reasoning from repetitive labor-intensive data exploration.

	Baseline	ORCA
Task Completion Rate	0.75	0.83
Discovery Success Rate	0.67	0.83
Data Selection Success Rate	0.75	1.00
CI coverage	0.083	0.50

Table 1: The table reports task completion rate, causal discovery success, data selection success, and confidence interval (CI) coverage across conditions. All metrics corresponds to better performance when they have higher values. ORCA consistently outperforms the baseline across all metrics, achieving higher task completion and discovery success, perfect data selection success, and substantially improved CI coverage, indicating more reliable causal effect estimation.

in Table 1, 50% of ORCA’s ATE estimates successfully fall within the confidence interval of the ground-truth effect, compared to 8% for the baseline. These improvements reflect cumulative gains from upstream stages, ORCA mitigating error propagation via step-wise pipeline that prevents users from skipping analytically required steps. The system prevents users from carrying forward unchecked assumptions and ensures consistency across stages, thereby directly shaping downstream feasibility and correctness.

Findings 2: Reallocation of user effort and reduced trial-and-error through structured interaction. ORCA reshapes how users interact with the causal analysis workflow by making early decisions explicit and surfacing errors earlier. Under the baseline condition, users spent up to 77% of their total analysis time to repeated data extraction (see Figures 5(a, d)), often leading to backtracking due to omitted variables or overly small subsets. In contrast, ORCA reduces exploration time by approximately 76% by allowing users to reallocate effort toward more critical stages such as assumption validation and result interpretation. This shift is accompanied by fewer corrective revisions in later stages. Interaction logs show that baseline users issued revision requests on average of 2.0 times

during causal discovery and 1.64 times during effect estimation, whereas ORCA users required fewer revisions, with averages of 1.1 and 0.71, respectively. Table 2 shows that, instead of repeatedly revising outputs, ORCA users spent modestly more time inspecting intermediate artifacts to confirm that each step had been completed as intended. Even with the more time spent in causal reasoning, total time spent on analysis was reduced by 21%. Survey responses corroborate these findings, with participants rating ORCA substantially higher in transparency (5.22 vs. 3.28) and trust (5.13 vs. 3.33), alongside reduced trial-and-error compared to the baseline (see Tables E.2, E.3

4 Conclusion and Future Work

We present ORCA, an interactive multi-agent framework for end-to-end causal analysis on relational databases. Motivated by limitations in existing LLM agent workflows, ORCA organizes analysis as a dependency-aware, step-wise workflow with shared state and human-verified checkpoints, preventing early specification errors from propagating downstream. Evaluations and a controlled user study show that this orchestration improves both efficiency and causal outcome quality, yielding lower ATE error and higher task completion rates. To support evaluation in realistic relational settings, we introduce REEF, a semi synthetic relational e-commerce benchmark with an explicit structural causal model. Future work includes exploring more flexible iteration across stages while preserving verifiability, evaluating ORCA with larger and more diverse user populations, and extending the framework to additional tasks and domains.

Acknowledgments

This work was supported by Institute of Information & communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (NoRS-2022-0-00612) This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (NoRS-2024-00410082) This

	User interaction time		User interaction turns	
	Baseline	ORCA	Baseline	ORCA
Data exploration step	439.34 $\pm$ 373.32	80.60 $\pm$ 77.71	5.72 $\pm$ 4.17	1 $\pm$ 2.96
Causal discovery step	78.49 $\pm$ 117.51	164.02 $\pm$ 155.64	2 $\pm$ 4.14	1.1 $\pm$ 2.331
Causal analysis step	71.48 $\pm$ 66.69	155.90 $\pm$ 202.15	1.64 $\pm$ 1.8	0.714 $\pm$ 1.25

Table 2: User interaction time and number of additional interaction turns across analysis stages for the baseline and ORCA conditions. ORCA substantially reduces user interaction time during data exploration while maintaining comparable or fewer interaction turns across all stages.

work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT)(NoNRF-2022M3J6A1063595)

References

- [1] Ragib Ahsan, David Arbour, and Elena Zheleva. 2023. Learning relational causal models with cycles through relational acycification. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 12164–12171.
- [2] Philipp Bach, Victor Chernozhukov, Malte S Kurz, and Martin Spindler. 2022. DoubleML—an object-oriented implementation of double machine learning in python. *Journal of Machine Learning Research* 23, 53 (2022), 1–6.
- [3] Zewude A. Berkessa, Esa Läärrä, and Patrik Waldmann. 2024. A Review of Causal Methods for High-Dimensional Data. *IEEE Access* (2024).
- [4] Iavor I. Bojinov, Albert Chen, and Min Liu. 2020. The Importance of Being Causal. *Harvard Data Science Review* 2, 3 (2020).
- [5] Gerlise Chan, Tom Claassen, Holger Hoos, Tom Heskes, and Mitra Baratchi. 2024. AutoCD: Automated Machine Learning for Causal Discovery Algorithms. In *AutoML 2024 Methods Track*.
- [6] Shuyu Dong, Kento Uemura, Akito Fujii, Shuang Chang, Yusuke Koyanagi, Koji Maruhashi, and Michèle Sebag. 2022. High-Dimensional Causal Discovery: Learning from Inverse Covariance via Independence-Based Decomposition. *arXiv:2211.14221*
- [7] Will Epperson, Gagan Bansal, Victor C Dibia, Adam Fourney, Jack Gerrits, Erkang Zhu, and Saleema Amershi. 2025. Interactive debugging and steering of multi-agent ai systems. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–15.
- [8] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large Language Model Based Multi-Agents: A Survey of Progress and Challenges. *arXiv:2402.01680*
- [9] Kairong Han, Kun Kuang, Ziyu Zhao, Junjian Ye, and Fei Wu. 2024. Causal Agent Based on Large Language Models. *arXiv:2408.06849*
- [10] James J. Heckman and Rodrigo Pinto. 2022. The Econometric Model for Causal Policy Analysis. *Annual Review of Economics* 14, 1 (2022), 893–923.
- [11] Paul W Holland. 1986. Statistics and causal inference. *Journal of the American statistical Association* 81, 396 (1986), 945–960.
- [12] Zhijiang Jin, Jiarui Liu, Zhiheng Lyu, Spencer Poff, Mrinmaya Sachan, Rada Mihalcea, Mona Diab, and Bernhard Schölkopf. 2023. Can large language models infer causation from correlation? *arXiv preprint arXiv:2306.05836* (2023).
- [13] Elahe Khatibi, Mahyar Abbasian, Zhongqi Yang, Iman Azimi, and Amir M Rahmani. 2024. Alcm: Autonomous llm-augmented causal discovery framework. *arXiv preprint arXiv:2405.01744* (2024).
- [14] F. Kühne, M. Schomaker, I. Stojkov, B. Jahn, A. Conrads-Frank, S. Siebert, G. Sroczynski, S. Puntischer, D. Schmid, P. Schnell-Inderst, and U. Siebert. 2022. Causal Evidence in Health Decision Making. *German Medical Science* 20 (2022), Doc12. doi:10.3205/000314
- [15] Mandar Kulkarni. 2025. Agent-s: LLM Agentic Workflow to Automate Standard Operating Procedures. *arXiv:2503.15520*
- [16] Sören R Künzel, Jasjeet S Sekhon, Peter J Bickel, and Bin Yu. 2019. Metalearners for estimating heterogeneous treatment effects using machine learning. *Proceedings of the national academy of sciences* 116, 10 (2019), 4156–4165.
- [17] LangChain. 2024. LangGraph Studio. <https://github.com/langchain-ai/langgraph>
- [18] Hao Duong Le, Xin Xia, and Zhang Chen. 2024. Multi-Agent Causal Discovery Using Large Language Models. *arXiv:2407.15073*
- [19] Jinyang Li, Binyuan Hui, Ge Qu, Binhua Li, Jiaxi Yang, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Chenhao Ma, Kevin C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLMs Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQL. *arXiv:2305.03111*
- [20] Marc Maier, Katerina Marazopoulou, David Arbour, and David Jensen. 2013. A sound and complete algorithm for learning causal models from relational data. *arXiv preprint arXiv:1309.6843* (2013).
- [21] Sakib A. Mondal, Prashanth Rv, and Sagar Rao. 2025. A Fast Algorithm for High-Dimensional Causal Discovery. *International Journal of Data Science and Analytics* 20 (2025), 1–10.
- [22] Matteo Negro, Andrea Piras, Ragib Ahsan, David Arbour, and Elena Zheleva. 2025. Relational Causal Discovery with Latent Confounders. *arXiv preprint arXiv:2507.01700* (2025).
- [23] Jonas Peters and Peter Bühlmann. 2015. Structural intervention distance for evaluating causal graphs. *Neural computation* 27, 3 (2015), 771–799.
- [24] Jonas Peters, Dominik Janzing, and Bernhard Schölkopf. 2017. *Elements of causal inference: foundations and learning algorithms*. The MIT press.
- [25] Babak Salimi, Harsh Parikh, Moe Kayali, Lise Getoor, Sudeepa Roy, and Dan Suciu. 2020. Causal relational learning. In *Proceedings of the 2020 ACM SIGMOD international conference on management of data*. 241–256.
- [26] Amit Sharma and Emre Kiciman. 2020. DoWhy: An End-to-End Library for Causal Inference. *arXiv:2011.04216*
- [27] Shohei Shimizu, Patrik O Hoyer, Aapo Hyvärinen, Antti Kerminen, and Michael Jordan. 2006. A linear non-Gaussian acyclic model for causal discovery. *Journal of Machine Learning Research* 7, 10 (2006).
- [28] Maojun Sun, Ruijian Han, Binyan Jiang, Houduo Qi, Defeng Sun, Yancheng Yuan, and Jian Huang. 2025. Lambda: A Large Model Based Data Agent. *J. Amer. Statist. Assoc.* (2025), 1–20.
- [29] Irene Testini, José Hernández-Orallo, and Lorenzo Pacchiardi. 2025. Measuring Data Science Automation: A Survey of Evaluation Tools for AI Assistants and Agents. *arXiv:2506.08800*
- [30] Ioannis Tsamardinos, Laura E Brown, and Constantin F Aliferis. 2006. The max-min hill-climbing Bayesian network structure learning algorithm. *Machine learning* 65, 1 (2006), 31–78.
- [31] Stefan Wager and Susan Athey. 2018. Estimation and inference of heterogeneous treatment effects using random forests. *J. Amer. Statist. Assoc.* 113, 523 (2018), 1228–1242.
- [32] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, and Jiale Liu. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *Proceedings of the First Conference on Language Modeling*.
- [33] Tongshuang Wu, Michael Terry, and Carrie Jun Cai. 2022. Ai chains: Transparent and controllable human-ai interaction by chaining large language model prompts. In *Proceedings of the 2022 CHI conference on human factors in computing systems*. 1–22.
- [34] Kun Zhang and Aapo Hyvärinen. 2012. On the identifiability of the post-nonlinear causal model. *arXiv preprint arXiv:1205.2599* (2012).
- [35] Yujia Zheng, Biwei Huang, Wei Chen, Joseph Ramsey, Mingming Gong, Ruichu Cai, Shohei Shimizu, Peter Spirtes, and Kun Zhang. 2024. Causal-learn: Causal discovery in python. *Journal of Machine Learning Research* 25, 60 (2024), 1–8.
- [36] Jialong Zhou, Lichao Wang, and Xiao Yang. 2025. GUARDIAN: Safeguarding LLM Multi-Agent Collaborations with Temporal Graph Modeling. *arXiv preprint arXiv:2505.19234* (2025).

A Related Works

A.1 Multi-Agent system and HITL

The rapid advancement of large-scale language models (LLMs) has shifted their use from single-model response generation to multi-agent systems that integrate tool usage and stateful interaction. These systems are increasingly realized as multi-agent frameworks, where multiple specialized agents collaborate to decompose and solve complex tasks. Representative frameworks such as AutoGen [32] and LangGraph [17] exemplify this paradigm by enabling structured interaction among agents with distinct roles. Despite these advancements, multi-agent systems inherit fundamental limitations of their underlying LLMs. Problems such as hallucination, and unstable reasoning are often amplified through cascading interactions, leading to error propagation across the workflow [8, 36]. As multi-agent systems complexity grows, mitigating such error accumulation has become a central research concern.

Moving beyond fully autonomous paradigms, emerging studies underscore the necessity of integrating human-AI interaction within multi-agent workflows [7]. Rather than passively observing agent outcomes, this new direction emphasizes interactive mechanisms that enable users to inspect system internal states, correcting erroneous trajectories, and dynamically steering analytical processes. These Human-in-the-Loop (HITL) mechanisms provide a foundation for more transparent and controllable multi-agent systems [33]. Core functionalities include pausing execution, rewinding to previous states, and editing prior messages to alter workflow trajectories, enabling localization of failure points and real-time verification of corrective interventions [7].

A.2 Automating Causal methods

Causal analysis plays a critical role in reliable decision-making by uncovering the underlying cause-effect mechanisms beyond mere correlations, and is widely recognized as essential for improving model interpretability across domains. Despite its importance, causal discovery and inference remain fundamentally challenging in real-world settings. Observational data alone makes it difficult to control for latent confounders and nonlinear dependencies, while the combinatorial explosion of potential directed acyclic graphs (DAGs) further increases computational complexity. Moreover, the performance of causal discovery algorithms varies considerably depending on data characteristics (continuous, discrete, or mixed types), and the process of method selection and hyperparameter tuning also requires substantial domain expertise. Consequently, recent research has increasingly explored LLM-based approaches and task-level automation to reduce expert burden in causal analysis (e.g. dowhy [26], causallearn [35]).

Traditional causal discovery methods include constraint-based, score-based, and functional causal model (FCM) approaches, but their performance varies widely across datasets and often requires manual configuration. To address these limitations, prior work on automated causal discovery has focused on reducing manual model selection and configuration. AutoCD [5] automated the selection and tuning of causal discovery algorithms via Bayesian Optimization, addressing performance variability across datasets. Parallel to these efforts, the integration of LLMs into causal reasoning workflows has attracted growing attention. The Causal

Agent [9] leverages LLMs to automate causal analysis on tabular data using external tools, demonstrating that LLMs can effectively decompose and analyze causal problems. Also, Multi-Agent Causal Discovery Framework (MAC) [18] employs multiple LLM agents to collaboratively generate and validate causal graphs through structured debate.

Collectively, these studies signify the emergence of an Agentic System paradigm for partially automating causal analysis tasks. This direction lays the foundation for next-generation causal reasoning systems that combine automation, adaptability, and human interpretability within a unified framework.

A.3 Causal Inference over Relational Databases

Causal inference over relational databases has become increasingly important as real-world data are stored across multiple interconnected tables; however, traditional causal analysis tools struggle to handle the complexity of such multi-relational structures [25]. Standard causal discovery methods typically assume a single i.i.d. dataset [11], making them ill-suited for relational settings where observations are not independent across tables.

To bridge this gap, recent work has proposed relational causal discovery algorithms that extend classical techniques to multi-table data. For example, Ahsan et al. [1] enable causal discovery in cyclic relational models by characterizing conditions under which RCD [20] remains sound despite mutual dependencies. In addition, Negro et al. [22] propose a relational FCI method (RelFCI) to account for latent confounders that span multiple linked tables.

In practice, causal analysis over relational databases further requires validation of modeling assumptions which are often implicit and error-prone in real-world data. However, even with existing advances in relational causal discovery, research on full workflow supporting causal analysis directly over relational databases remains an open and active area.

B Implementation Details of ORCA framework

This section provides implementation-level details of the ORCA framework to support reproducibility. We describe the concrete inputs, outputs, prompts, diagnostics, and control logic used by each agent, complementing the high-level system description presented in Section 2. Unless otherwise noted, this section focuses on how each component is instantiated in practice rather than on design motivations, which are discussed in the main text. To support transparency and reproducibility, we release the full implementation of ORCA, including all agents and orchestration logic.¹

B.1 Details for the Data Exploration Agent

B.1.1 Table Explorer. Given a user-specified table, *Table Explorer* operates over database metadata, including schema definitions, column-level statistics, and foreign key relationships. Using the prompt shown in Prompt F.1, it summarizes table- and column-level properties such as null ratios, distinct counts, and distributional characteristics. These summaries surface constraints on how columns can be used in analysis. For example, columns with near-zero variance are flagged as invalid candidates for treatments or outcomes, while columns with high missingness flags potential

¹<https://anonymous.4open.science/r/ORCA-E70F>

data quality concerns. The module further examines the table’s relational context by identifying adjacent tables. This step allows users to anticipate how joins may change the unit of analysis or introduce duplication. Finally, the *Table Explorer* synthesizes statistical and relational signals into an explicit assessment of data suitability, highlighting integrity issues and structural risks relevant to subsequent steps. An example of the output is shown in Figure G.1.

B.1.2 Table Recommender. *Table Recommender* takes as input a natural language query or analysis plan, together with database metadata which includes schema information and foreign key relationships, and diagnostics from the *Table Explorer*. Guided by Prompt F.2, it first extracts an analytical objective by identifying target entities, outcomes of interest, and required information types (e.g., behavioral, transactional, or temporal). It then narrows the search space via semantic matching over table and column metadata to identify candidate tables relevant to the analysis and filter out unrelated tables. Finally, the module explores foreign key paths among candidate tables to assess relational connectivity. This analysis verifies that necessary join paths exist and makes explicit how tables are connected. As a result, users can identify relational paths that may introduce confounding, as well as tables that may lead to unnecessary joins. Based on these steps, the *Table Recommender* produces a ranked list of tables and an ERD visualizing their connections, helping users pinpoint database components essential to their research goals and validate the relational scope before downstream analysis. See Figure G.2 for detailed output.

B.1.3 Text2SQL Generator. *Text2SQL Generator* takes as input the user query, the set of tables selected by the *Table Recommender*, and database metadata. Guided by the generation prompt shown in Figure F.3, it first parses the natural language request into a structured query specification, identifying target entities, variables of interest, temporal constraints, and the intended unit of analysis. Based on this specification, it builds a query skeleton by determining appropriate SELECT, JOIN, WHERE, and GROUP BY clauses, while restricting joins to the recommended tables and valid schema relationships. To ensure robustness, the module incorporates an automated execution-feedback loop, which iteratively corrects errors in column references, join conditions, or aggregation logic via a self-correction prompt (Figure F.4). The module further validates whether the executed result aligns with the original analytical intent using an intent-validation prompt (Figure F.5), preventing cases where a query is technically valid but semantically misaligned. This structured and automated refinement reduces the need for manual trial-and-error. The Text2SQL Generator ultimately returns an executable SQL query along with its resulting dataset. This output provides a well-defined and verifiable dataset specification for subsequent causal discovery and causal inference. An example execution trace and generated output are shown in Figure G.3.

B.1.4 Data Preprocessor. *Data Preprocessor* applies data transformation including outlier handling, normalization, and format alignment to produce clean, analysis-ready datasets. As part of this process, it performs schema detection to annotate variable information. Specifically, the module infers data types and computes categorical cardinalities, providing the specifications required for

Causal Discovery. This module serves as a final checkpoint before modeling, ensuring that analytical assumptions align with the prepared dataset.

B.2 Details for the Causal Discovery Agent

B.2.1 Testing Causal Hypothesis. The first step performs a comprehensive statistical diagnosis to assess the validity of key causal assumptions underlying different discovery algorithms. To this end, the module is organized as a series of diagnostic tests that evaluate data properties relevant to these assumptions. This step produces a structured data profile that summarizes assumption-relevant properties of the dataset through three hierarchical checks:

- (1) **Basic Checks.** The dataset is first classified as *Pure Continuous*, *Pure Categorical*, or *Mixed*. Then the reliability of Conditional Independence (CI) tests is assessed by comparing the sample size (n) to the number of variables (p); when $p > n$, CI reliability is flagged critically low, prompting conservative algorithm choices. High-cardinality categorical variables are also detected to avoid computational bottlenecks in discrete or mixed-type algorithms.
- (2) **Global Tests.** Dataset-level assumptions are evaluated using multivariate tests. For continuous data, the Henze–Zirkler (HZ) test is applied to assess multivariate normality, where $p < 0.05$ indicates deviation from Gaussianity. To evaluate global linearity, the Ramsey RESET test is performed on continuous variables and one-hot encoded mixed variables.
- (3) **Pairwise Profiles:** This check analyzes all pairs of continuous variables.
 - **Linearity Score (S_{lin})** For each variable pair, linearity is tested via a nested comparison between a linear model and a spline-based Generalized Additive Model (GAM). Let p_{nonlin} denote the p-value for the smooth term; a linearity indicator is defined as $\mathbb{I}_{lin} = 1(p_{nonlin} > \alpha)$ with $\alpha = 0.05$.
 - **Non-Gaussianity Score (S_{ng})** For each pair, we test residual normality using the Jarque–Bera and Shapiro–Wilk tests. Let p_{val} be the resulting p-value and define $S_{ng} = 1 - p_{val}$, where larger values indicate stronger evidence of non-Gaussian noise.
 - **ANM Score:** Consistency with the Additive Noise Model is assessed by testing independence between predictors and residuals using HSIC. The resulting p-value is used as S_{anm} , with higher values indicating stronger support for the ANM assumption.

For the linearity score, pairwise binary indicators are aggregated by their mean, yielding the proportion of variable pairs consistent with linearity. For non-Gaussianity and ANM scores, pairwise values are aggregated using the median to ensure robustness against unstable or weakly related variable pairs.

B.2.2 Algorithm Configuration. Based on the data profile, the *Algorithm Configuration* module determines a tailored execution plan. This step maps statistical properties to a portfolio of compatible algorithms, ensuring that each solver is applied only when its underlying assumptions are satisfied. The agent evaluates properties such as data type, cardinality, and the aggregated linearity and Gaussianity scores to determine the appropriate configuration, following the branching logic illustrated in Figure B.1. Thresholds on

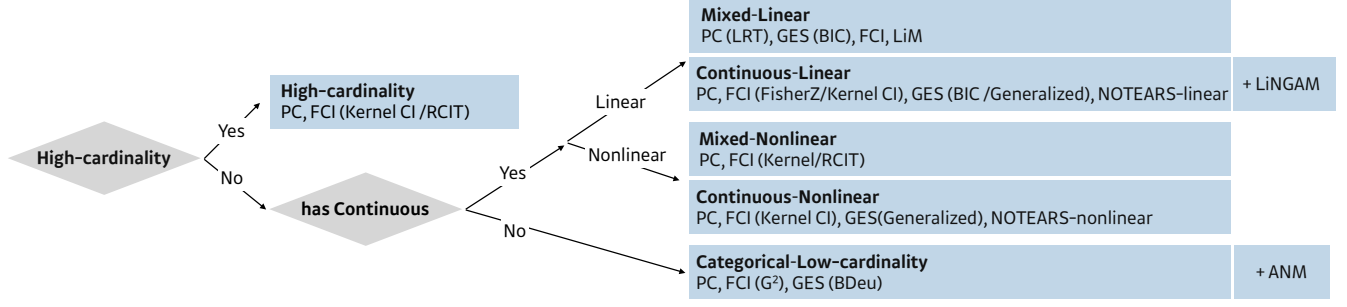


Figure B.1: Decision-tree of ORCA's algorithm configuration. The tree selects a base causal discovery scenario based on categorical cardinality, the data type(presence of continuous variables), and linearity diagnostics. Leaf nodes summarize compatible algorithm families, with additional assumption checks to conditionally add optional methods (LiNGAM and ANM).

aggregated diagnostic scores are applied as implementation-level criteria. Specifically, we use a fixed threshold $\tau_{lin} = 0.5$ on the aggregated linearity score, corresponding to a majority of variable pairs exhibiting linear behavior. Similarly, $\tau_{ng} = \tau_{anm} = 0.5$ are used as majority-based thresholds for non-Gaussianity and additive noise diagnostics, which guide the conditional inclusion of additional discovery methods. These values were fixed by selecting the setting that achieved the lowest and most stable SHD/SID among candidate thresholds.

B.2.3 Graph Generation and Scoring. The third step generates candidate causal graphs according to the execution plan and evaluates their plausibility using stability and consistency based criteria. The execution module runs the algorithms specified in the plan in parallel on the preprocessed data. The output is a set of candidate graphs $G = \{G_1, G_2, \dots, G_k\}$, which may include DAGs, CPDAGs, or PAGs. Because different algorithms rely on distinct optimization criteria and scoring functions, their outputs are not directly comparable. Instead, ORCA evaluates each candidate graph based on its agreement with the observed data distribution and its stability under perturbations, using a composite score derived from three metrics:

- (1) **Markov Consistency (S_{mc}):** This metric measures whether the conditional independence relations implied by a graph are supported by the data. For a given graph G , we extract a local Markov basis of implied d-separation statements and test each statement. The score is defined as

$$S_{mc}(G) = 1 - \frac{N_{\text{rejected}}}{N_{\text{tested}}}, \quad (1)$$

where N_{rejected} denotes the number of conditional independence hypotheses rejected by the data at significance level α , and N_{tested} is the total number of tested statements. For graph types that do not encode a full set of testable conditional independencies (e.g., PAGs), this metric is omitted.

- (2) **Sampling Stability (S_{samp}):** To assess robustness to sampling variability, we apply bootstrap resampling. Given B bootstrap samples, the same discovery algorithm is re-run to obtain an ensemble $\{G_1^*, \dots, G_B^*\}$. The confidence of an edge e is defined as its empirical frequency $P(e) = \frac{1}{B} \sum_{b=1}^B \mathbb{I}(e \in G_b^*)$.

The sampling stability score of graph G is the average confidence over its edges:

$$S_{\text{samp}}(G) = \frac{1}{|E_G|} \sum_{e \in E_G} P(e). \quad (2)$$

- (3) **Structural Stability (S_{struct}):** This metric evaluates sensitivity to variable selection. We generate random subsets of variables $V_i \subset V$, re-run the algorithm on each subset to obtain G_i , and compare it with the restriction of the original graph $G|_{V_i}$. Instability is quantified using a graph-type-appropriate Structural Hamming Distance (SHD) [30]. For partially oriented graphs (e.g., CPDAGs), SHD is computed on adjacencies only.

The final **Composite Score** is computed as a weighted sum of the applicable metrics:

$$\text{Score}(G) = w_1 S_{mc} + w_2 S_{\text{samp}} + w_3 S_{\text{struct}} \quad (3)$$

where the weights are normalized over the metrics applicable to the graph type. Candidate graphs are ranked by this score, and the top- k graphs are forwarded to the synthesis step. In our implementation, we use fixed weights $(w_1, w_2, w_3) = (0.4, 0.3, 0.3)$. These weights balance consistency with the observed data and stability under perturbations, and are kept fixed across all experiments.

B.2.4 Ensemble and Synthesis. The final step integrates multiple candidate causal graphs into a unified representation by aggregating structural evidence and resolving directional conflicts.

Consensus PAG Construction. ORCA first builds a consensus skeleton by identifying robust adjacencies shared across candidate graphs. For each variable pair (X, Y) , an undirected confidence score is computed by aggregating the normalized composite scores of candidate graphs that contain the corresponding adjacency. If the aggregated confidence exceeds a predefined threshold τ (e.g., $\tau = 0.5$), the adjacency is retained. Given the retained adjacencies, ORCA derives a PAG-like intermediate graph. For each adjacency (X, Y) , directional evidence is aggregated using weighted voting over the top- k candidate graphs. Let w_i denote the normalized composite score of graph G_i . Only explicitly directed edges ($\rightarrow$) contribute to the vote. If weighted support for one direction exceeds that of the opposite direction, the edge is oriented accordingly;

otherwise, it is marked as uncertain (e.g., $X \circ - \circ Y$). The resulting `consensus_pag` preserves both robust structural relations and directional uncertainty where the ensemble does not provide sufficient agreement.

Actionable DAG Synthesis. To enable downstream causal inference, ORCA converts the consensus PAG into a single directed acyclic graph (DAG). Edges are processed in descending order of confidence and added incrementally under an explicit acyclicity constraint. Edges with resolved directions are adopted whenever they do not introduce cycles. For uncertain or conflicting edges, ORCA applies a deterministic, rank-consistent tie-breaking strategy: the direction proposed by the highest-ranked candidate graph that specifies an explicit orientation is selected. If this choice introduces a cycle, the opposite orientation is attempted; if both violate acyclicity, the edge is discarded. If no candidate provides directional evidence, ORCA attempts a cycle-safe orientation; otherwise, the edge is dropped. The resulting graph is guaranteed to be fully directed and acyclic.

As a result, the **Causal Discovery Agent** outputs two complementary artifacts: a `consensus_pag` that preserves structural uncertainty for inspection, and a fully directed `final_dag` used for downstream causal effect estimation. An example execution log is shown in Figure G.4.

B.3 Details for the Causal Inference Agent

B.3.1 Variable Selection. The goal of the *Variable Selection* module is to parse the user’s question as a causal query by fixing the meaning of the treatment(T) and outcome(Y) with respect to the available data. This step ensures that subsequent identification and estimation strategies operate over well-defined variables and reduces failures caused by mismatches between user intent and data representation. Given the user query, the module utilizes Prompt F.6 to select treatment and outcome variables using the available columns, a schema summary, and a sample of the data. Using the selected causal graph, remaining variables are then assigned causal roles according to their structural relationships to T and Y , including confounders (common causes of T and Y), mediators (post-treatment variables on causal paths from T to Y), valid instruments, and variables that should not be conditioned on, such as colliders or descendants of the outcome.

B.3.2 Configuration Selection. The *Configuration Selection* module selects one of several predefined tasks, employing Prompt F.7 that lists valid options with concise descriptions. Then, the identification strategy is selected from backdoor, frontdoor, instrumental variable, mediation, or the ID algorithm. Candidate confounders, mediators, and instruments identified in the previous stage inform this choice. For example, when no valid instrumental variables are present, IV-based strategies are excluded. Estimator selection is guided by the outcome type and data characteristics; for instance, binary outcomes favor generalized linear models, while continuous outcomes favor linear regression-based estimators.

B.3.3 Model Implementation. The *Model Implementation* module first converts the selected causal graph into a DoWhy-compatible representation. It then checks basic structural requirements implied by the chosen identification strategy, such as the existence of a

directed path from the treatment to the outcome or the validity of candidate instruments. If these requirements are not satisfied, the module records and reports the reason for failure. When the requirements are satisfied, the module constructs a causal model, identifies the estimand, and further applies the selected estimator and refutation when specified. If refutation methods are specified, DoWhy refuters are executed and their outcomes are summarized. The resulting model object, estimand, and estimate with confidence intervals are stored. If estimation fails, the failure and its cause are explicitly reported. Optional fallback results (e.g., regression-based adjustments) are provided only for descriptive references and are distinguished from causal estimates.

B.3.4 Interpretation. *Interpretation* module uses an LLM with Prompt F.8 to generate an understandable explanation of causal inference results based on the outputs of previous stages. Rather than merely verbalizing numerical estimates, it summarizes the assumptions, identification conditions, and limitations that govern how the result should be interpreted. By making these conditions explicit, the module helps users assess the credibility and scope of the causal claim. A sample output is provided in log G.5.

C REEF

C.1 Database Schema and Relational Structure

REEF is implemented as a relational PostgreSQL database designed to reflect the structure of real-world e-commerce systems. The schema consists of 18 tables and 130 variables spanning user profiles, product catalogs, transactional processes, and post-purchase interactions. These tables are connected through explicit foreign-key relationships, inducing multi-hop relational paths (e.g., $\text{user} \rightarrow \text{order} \rightarrow \text{payment}$, or $\text{category} \rightarrow \text{brand} \rightarrow \text{product}$), which are commonly encountered in enterprise analytics. This design mirrors operational database practices and encourages evaluation under realistic analytical workflows, where relevant features must be explicitly defined through joins rather than directly observed. Figure C.1 illustrates the overall relational structure and causal relationships of variables.

C.2 Data Generation and Seeding Process

REEF is populated using a semi-synthetic data generation pipeline that combines domain-informed structural equations with stochastic noise. Data seeding is performed in a fixed order that respects causal and relational dependencies, starting from exogenous entities (e.g., categories, users) and progressing toward downstream transactional tables (e.g., cart, orders, payment, shipping). Each table is generated using a combination of rule-based logic and probabilistic sampling implemented in `Faker.js`. For example, user attributes such as age, gender, and browsing behavior are sampled from biased distributions to reflect realistic population characteristics, while latent activity scores are computed through explicit functional relationships and transformed into binary activity indicators via logistic functions. Downstream processes, including cart quantities, order composition, discount usage, and payment outcomes, are generated conditional on upstream variables, ensuring that observed correlations arise from known causal mechanisms rather than arbitrary randomness. Importantly, the seeding process

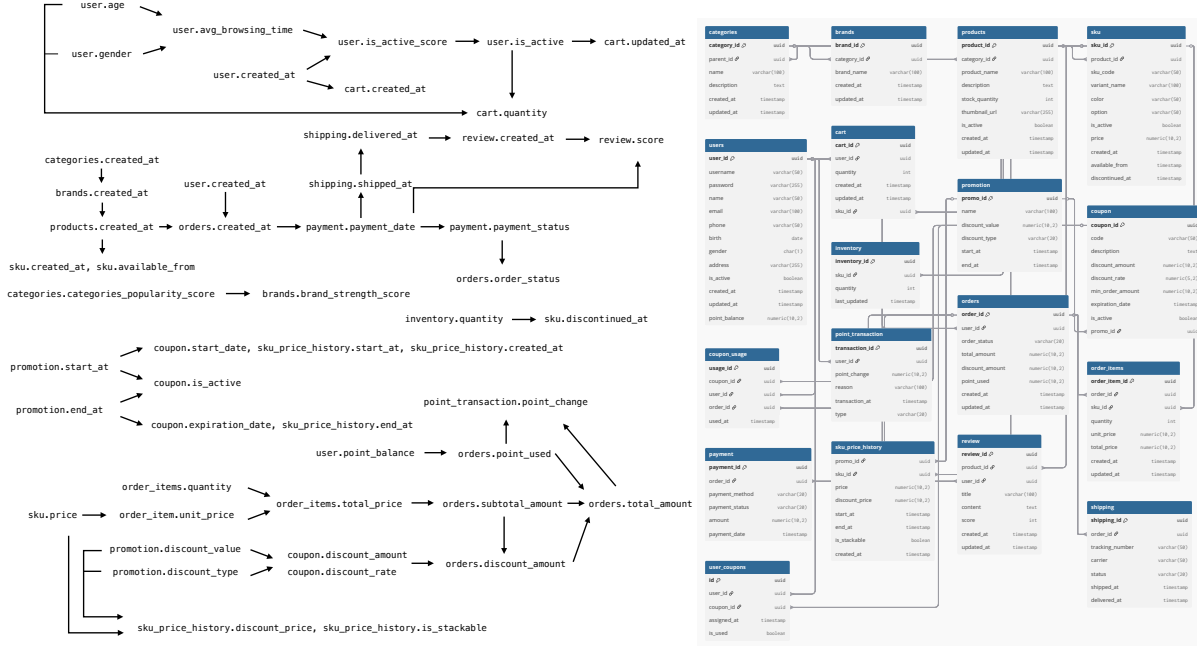


Figure C.1: Causal graph and relational schema of the REEF benchmark. The left panel shows the ground-truth structural causal model (SCM) used to generate the data, while the right panel presents the corresponding entity–relationship diagram (ERD) of the relational database. Variables relevant to a single causal query are distributed across multiple tables and linked through temporal and relational dependencies.

injects controlled noise, missing values, and temporal variability (e.g., delayed payments, inactive users, discontinued products), preserving the irregularities of observational data while maintaining a known underlying data-generating process.

C.3 Design Choices and Limitations

REEF is designed to balance realism and causal transparency. Rather than generating fully synthetic tabular data, we adopt a relational, event-driven design that mirrors operational databases used in practice. This choice enables evaluation of causal methods under realistic challenges such as multi-table joins, aggregation-induced bias, and mixed variable types. At the same time, several simplifying assumptions are made. First, while temporal ordering is respected in data generation, REEF does not model unobserved time-varying confounders beyond those explicitly encoded in the structural equations. Second, some treatments (e.g., categorical product attributes) require aggregation or encoding choices when used for effect estimation, which may introduce modeling ambiguity. Finally, although missingness is intentionally introduced, it follows controlled mechanisms and does not capture all forms of non-random data absence observed in real deployments. Despite these limitations, REEF provides a practical and reproducible benchmark that preserves the essential complexity of relational observational data while enabling rigorous evaluation against known causal ground truth.

D Evaluation of Specialist Agents

We conduct a series of experiments to evaluate the independent robustness of each specialist agents.

D.1 Evaluation Metrics

This section provides formal definitions of the evaluation metrics used throughout the experiments.

Structural Hamming Distance (SHD). SHD [30] measures the structural discrepancy between a predicted graph $\hat{G}$ and the ground-truth graph G . It is defined as the minimum number of edge insertions, deletions, or reversals required to transform $\hat{G}$ into G :

$$\text{SHD}(\hat{G}, G) = |\{(i, j) \in V^2 \mid \hat{G}_{ij} \neq G_{ij}\}|. \quad (4)$$

Lower SHD indicates closer structural agreement.

Structural Intervention Distance (SID). SID [23] evaluates the discrepancy between two graphs in terms of implied interventional distributions. Formally, SID counts the number of variable pairs for which the predicted graph yields incorrect interventional effects compared to the ground-truth graph. Lower SID indicates higher causal consistency under interventions.

Precision and Recall (Table Retrieval). For table retrieval, precision and recall are computed by comparing the set of tables recommended by the system R_n with the ground-truth relevant tables

G_n :

$$\text{Recall} = \frac{1}{N} \sum_{n=1}^N \frac{|G_n \cap R_n|}{|G_n|}, \quad \text{Precision} = \frac{1}{N} \sum_{n=1}^N \frac{|G_n \cap R_n|}{|R_n|}. \quad (5)$$

Execution Accuracy (EX). For text-to-SQL generation, Execution Accuracy measures whether the predicted SQL query produces the same execution result as the ground-truth query when run against the database:

$$\text{EX} = \frac{1}{N} \sum_{n=1}^N \mathbb{I}(\hat{V}_n = V_n). \quad (6)$$

Treatment Effect Metrics. For treatment effect estimation, we report: (i) absolute error of the estimated Average Treatment Effect (ATE), (ii) bias (values closer to zero indicate better performance), (iii) confidence interval (CI) coverage, and (iv) CI width. CI coverage is defined as the proportion of experiments in which the true effect lies within the estimated confidence interval.

D.2 Data Exploration Agent

Dataset. We use REEF and BIRD [19] data to evaluate **Data Exploration Agent**. BIRD is a text-to-SQL benchmark grounded in real-world domains, designed to test models under challenging conditions such as external knowledge reasoning, SQL efficiency over large databases, and complex query structures including implicit joins and aggregations. For reproducibility and efficient error analysis, we adopt the official BIRD mini-dev subset, which includes 500 natural language queries across 12 diverse databases².

D.2.1 Table Description Evaluation. We evaluate the ability of *Table Explorer* to generate informative and concise natural language descriptions of database tables. For each table, both the baseline and ORCA are tasked with producing a description that conveys the core semantics and structure of the table. The quality of these descriptions is then assessed using a standardized rubric.

The baseline model is prompted to take a table as input and to create a description. Due to the large size of the tables, only the first 100 rows are used as input for baseline model. The evaluation is conducted using a 5-point scale rescaled to 0–100 for reporting, which measures how informative and meaningful each description is. We note that GPT-4o-mini is used as an automatic evaluator, which may introduce model-based evaluation bias. Accordingly, the reported scores are intended for relative comparison between methods under identical conditions, rather than as absolute measures of quality. To ensure fairness, GPT-4o-mini is prompted with clear and consistent grading instructions. Detailed grading criteria are provided in the Appendix F.

As shown in Figure D.1(a), ORCA significantly outperforms the baseline in both datasets, achieving average scores of 91.0 and 92.3 compared to 58.0 and 62.3, respectively. This demonstrates ORCA’s strong ability to interpret and summarize structured data effectively.

D.2.2 Table Retrieval Evaluation. We also evaluate the ability of the *Table Recommender* to recommend relevant tables given a natural language question. The goal of this task is to identify which tables from a database are necessary to answer a user query. The baseline model (GPT-4o-mini) is prompted with a list of all table names

in the database, followed by the user question. It is instructed to select which tables would be helpful for answering the question. We evaluate table retrieval using Recall and Precision, measuring coverage of ground-truth tables and correctness of recommended tables, respectively. As shown in Figure D.1 (b), ORCA consistently outperforms baseline on both datasets, achieving higher Recall and Precision. This indicates that ORCA is more effective at identifying relevant tables while minimizing the inclusion of unnecessary ones.

D.2.3 Text2SQL Generator Evaluation. We evaluate the ability of *Text2SQL Generator* to generate a valid SQL query for the user’s request. Each SQL prediction is executed against the corresponding database, and the output is compared with that of the ground-truth SQL to compute execution accuracy. Execution Accuracy (EX) measures the percentage of evaluation examples where the predicted SQL and the ground-truth SQL produce the same execution result. As shown in Figure D.1 (c), ORCA consistently outperforms the baseline across all difficulty levels on both datasets. Notably, while the REEF dataset reflects a more realistic and structurally complex database environment, ORCA shows a strong execution accuracy of 60.00%, whereas GPT-4o mini’s performance significantly drops to 6.67%.

D.2.4 Comparison with Frontier LLMs. To verify that ORCA’s improvements are not simply replicated by adopting a stronger backbone LLM, we evaluate GPT-5 and Gemini-2.5-Flash on all three Data Exploration tasks using the REEF dataset. For fair comparison, the same model is used as both the standalone baseline and ORCA’s internal LLM. As shown in Table D.1, frontier baselines improve Table Explorer scores over GPT-4o-mini (see Fig. D.1), yet ORCA achieves a perfect score with both backbones. For Table Recommender, ORCA shows stronger recall, suggesting it is more reliable at surfacing all relevant tables for a given query. On Text2SQL, ORCA consistently outperforms each corresponding baseline, though absolute gains remain modest given the inherent difficulty of SQL generation on complex schemas. Together with the causal discovery results in Table D.3, these findings indicate that ORCA’s agentic workflow adds value beyond what stronger LLMs alone provide.

D.3 Causal Discovery Agent

Dataset. We evaluate causal discovery performance using synthetic datasets generated from random directed acyclic graphs (DAGs) under controlled data-generating processes. As summarized in Table D.2, we consider five scenarios that vary graph topology (Erdős–Rényi vs. scale-free), SCM functional form (linear vs. nonlinear), and noise distribution (Gaussian vs. non-Gaussian). Given a ground-truth DAG, data are generated in topological order using additive SCMs, where each variable is computed as a normalized sum of its parent variables followed by either a linear transformation or a nonlinear activation, and additive noise. Noise terms are sampled independently from either a Gaussian or Laplace distribution, depending on the scenario. Across all settings, the number of variables is varied as $d \in \{3, 5, 10\}$, and sample sizes are chosen such that $N > D$. For each scenario and dimensionality, we generate 10 independent datasets using different random seeds.

²https://github.com/bird-bench/mini_dev

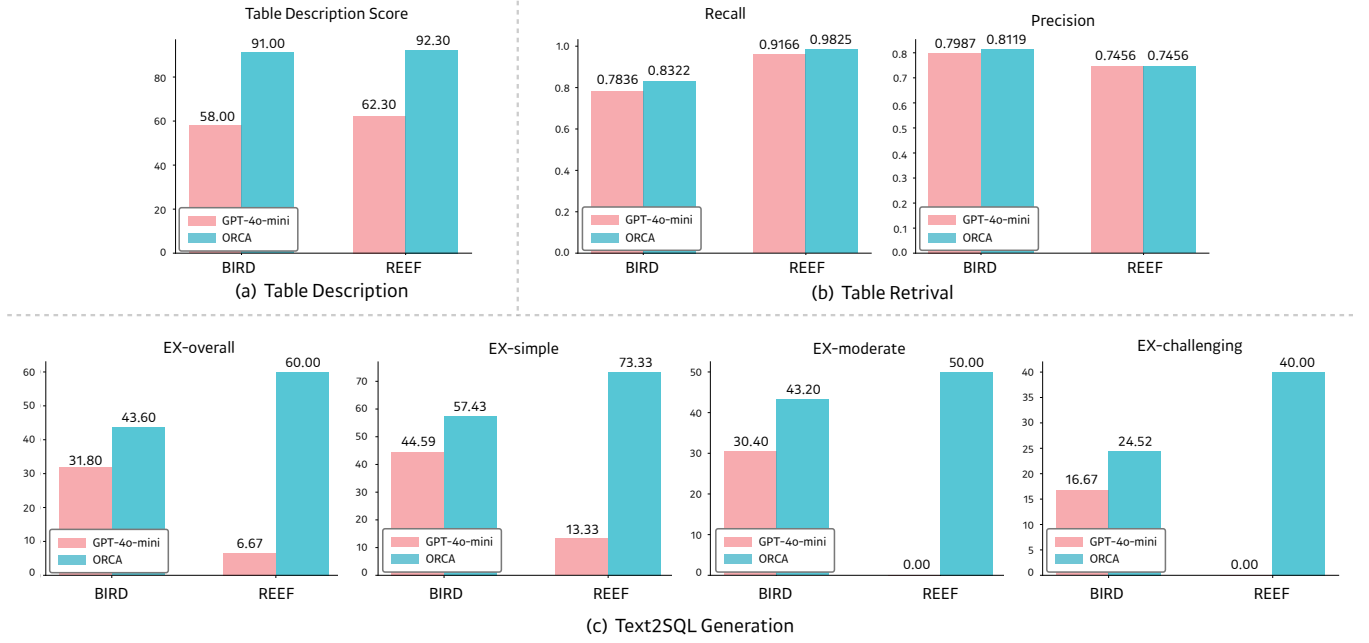


Figure D.1: Evaluation results of the Data Exploration Agent across multiple subtasks. The figure compares ORCA and GPT-4o-mini on table description, table retrieval, and Text2SQL generation using the BIRD and REEF benchmarks. ORCA consistently outperforms the baseline across all subtasks, with particularly large gains on the relationally complex REEF dataset and harder Text2SQL query categories.

Backbone LLM	Method	Table Explorer	Table Recommender			Text2SQL
		Quality Score $\uparrow$	Precision $\uparrow$	Recall $\uparrow$	F1 $\uparrow$	EX $\uparrow$
GPT-5	Baseline	91.2	0.766	0.868	0.796	0.467
	ORCA	100.00	0.599	1.000	0.723	0.500
Gemini-2.5-Flash	Baseline	82.2	0.730	0.807	0.748	0.433
	ORCA	100.00	0.754	0.842	0.770	0.467

Table D.1: Data Exploration module performance on the REEF dataset with frontier LLMs. For fair comparison, the same backbone LLM is used both as the standalone baseline and as ORCA’s internal model.

Scenario	Graph	SCM	Noise	Scale
Baseline (ER–Linear, Gaussian)	ER	Linear	Gaussian	$N > D$
Graph Complexity (Scale-Free–Linear)	SF	Linear	Gaussian	$N > D$
Noise Misspecification (ER–Linear, Non-Gaussian)	ER	Linear	Non-Gaussian (Laplace)	$N > D$
Structural Misspecification (ER–Nonlinear)	ER	Non-linear (e.g., MLP)	Gaussian	$N > D$
Fully Challenging Setting (Scale-Free–Nonlinear)	SF	Non-linear	Non-Gaussian	$N > D$

Table D.2: Synthetic causal discovery scenarios.

Setting. Given observational data sampled from a known ground-truth DAG, each method estimates a directed causal graph. We compare ORCA against two baselines: AutoCD, a classical automated causal discovery framework, and GPT-4o-mini, an LLM-based baseline that directly infers causal relations from data summaries; and

two frontier LLMs, GPT-5 and Gemini-2.5-Flash, evaluated under the same prompting protocol as GPT-4o-mini. All methods are evaluated under identical data-generating conditions, and performance is averaged across repeated runs.

Results. Table D.3 reports causal discovery performance across synthetic scenarios. In most settings, ORCA achieves lower structural errors than all baselines. Notably, while frontier models occasionally improve over GPT-4o-mini on individual scenarios (e.g., Gemini-2.5-Flash on ER–Linear at $d=10$), ORCA achieves the most consistent gains across graph topologies and functional forms. In scale-free graphs, which better reflect real-world causal sparsity patterns, ORCA achieves large and consistent improvements over all baselines. Also, ORCA shows robustness in performance even with noise perturbations or complex dependency structures. These results indicate that ORCA successfully guides causal discovery across varying graph topologies and functional forms, especially in settings where structural assumptions are difficult to specify a priori. For higher-dimensional setting with $d = 10$ variables, we restrict this analysis to three representative scenarios due to the rapidly increasing search space: ER–Linear, Scale-Free–Linear, and ER–Nonlinear. In Scale-Free–Linear and ER–Nonlinear scenarios, ORCA achieves lower SHD and SID than both baselines, indicating improved resilience to increased graph complexity and nonlinear mechanisms even in higher dimensions. AutoCD outputs do not directly support SID computation due to the absence of a fully specified directed graph; therefore, SID is omitted for this method.

D.4 Causal Inference Agent

Dataset. To evaluate treatment effect estimation performance of the Causal Inference Agent, we use both synthetic data tables and REEF with complex, multi-table data structures. Synthetic datasets are designed to evaluate robustness to confounding, model misspecification, and heterogeneous treatment effects. As summarized in Table D.4, we consider four scenarios that vary the functional form of treatment assignment, outcome generation, and the presence of heterogeneous treatment effects. Confounders are sampled from a standard normal distribution, and treatment is assigned via a logistic propensity model that is either linear or nonlinear in the confounders. Potential outcomes are generated using additive outcome models with either constant(ATE) or confounder-dependent (CATE) treatment effects, and the observed outcome is constructed according to the assigned treatment. Nonlinear outcome scenarios introduce misspecification through higher-order confounder terms. All treatment effect datasets use a fixed sample size ($N = 2000$) and number of confounders ($D = 5$), and each scenario is generated with multiple random seeds.

Setting. To distinguish estimation error caused by incorrect causal structure from error arising from statistical estimation, we explicitly use two experimental settings. In the Oracle Graph setting, estimators are provided with the ground-truth causal graph. This setting isolates the pure estimation capability of each method, independent of any error in causal discovery. In the Agentic Graph setting, effect estimation is performed using causal graphs inferred by ORCA or baseline discovery methods, allowing to assess how discovery-stage errors propagate to downstream effect estimation. For Oracle Graph settings, we compare ORCA with standard baselines including DoubleML [2], Causal Forest [31], and the T-learner [16]. The Agentic Graph setting applies only to methods that output explicit causal graphs; accordingly, this setting evaluates effect estimation using graphs inferred by ORCA.

Results. Table D.5 summarizes treatment effect estimation results across four synthetic scenarios under the oracle graph setting. Across all scenarios, ORCA consistently achieves the lowest ATE absolute error, ranging from 0.0030 in the linear baseline to 0.0077 in the heterogeneous treatment effect setting, outperforming all baseline estimators. In the non-linear outcome setting, ORCA remains stable while competing methods exhibit degraded coverage. ORCA also maintains high CI coverage (80–100%) while producing relatively narrower confidence intervals.

Table D.6 reports results on the REEF benchmark. Under the oracle graph setting, ORCA attains near-zero ATE error (3.2×10^{-5}) with 100% CI coverage, significantly outperforming strong baselines such as DoubleML and Causal Forest. When causal graphs are inferred rather than provided, estimation accuracy degrades as expected due to error propagation(e.g. missing confounders) from the discovery stage. Nevertheless, under the agentic graph setting, ORCA preserves high CI coverage and success rate on REEF, despite increased ATE error, demonstrating robustness to graph uncertainty.

E User Study Details

E.1 Detailed Study Protocol

This appendix provides the full study protocol used to evaluate ORCA against a LLM-assisted coding baseline for end-to-end causal analysis on a relational database benchmark (REEF). We focus on a single relational benchmark (REEF) to ensure a controlled causal setting with known ground-truth effects, allowing performance differences to be attributed to the analysis workflow rather than dataset-specific confounds. In the absence of a formal institutional review process, we followed internationally accepted ethical principles, including informed consent, voluntary participation, and data anonymization. All participants were informed about the study purpose and data usage and provided consent prior to participation. Participants were informed that interaction logs would be collected for research purposes, and consented to this data collection as part of the study agreement. The study was conducted remotely and followed a within-subjects design in which each participant completed both conditions. To mitigate order effects, participants were randomly assigned to one of two counterbalanced groups: (i) Baseline $\rightarrow$ ORCA, or (ii) ORCA $\rightarrow$ Baseline.

E.1.1 Timeline and Phases. The session was designed to take approximately 120 minutes in total, consisting of three phases:

- **Phase 1: Pre-setting (20 min)** Participants received study instructions and a short tutorial covering database setup and how to operate each system (Baseline and ORCA), including how to save and submit logs. Participants then completed a pre-study survey (approximately 5 minutes) assessing demographics, programming experience (Python/SQL), debugging familiarity, prior experience with LLM-assisted coding tools, and causal inference knowledge.
- **Phase 2: Main Sessions (up to 60 min $\times$ 2)** Participants completed two analysis tasks under two different conditions (Baseline vs. ORCA), following the assigned counterbalanced order. Each task had a strict time limit of **60 minutes**. If the

Scenario	Method	$d = 3$		$d = 5$		$d = 10$	
		SHD ↓	SID ↓	SHD ↓	SID ↓	SHD ↓	SID ↓
ER-Linear	AutoCD	3.0	–	10.0	–	41.4	–
	GPT-4o-mini	3.2	4.4	11.3	15.8	40.8	81.1
	GPT-5.2	3.4	4.6	10.5	16.2	40.4	79.1
	Gemini-2.5-Flash	3.5	4.4	9.4	14.5	39.8	76.8
	ORCA	1.9	2.1	10.8	15.1	44.2	83.3
Scale-Free-Linear	AutoCD	2.0	–	4.2	–	25.4	–
	GPT-4o-mini	3.3	4.2	8.0	11.5	24.8	67.2
	GPT-5.2	2.4	3.6	6.0	10.9	27.6	68.8
	Gemini-2.5-Flash	2.4	3.6	7.7	14.4	29.5	66.8
	ORCA	0.2	0.3	1.1	2.0	21.4	58.6
ER-Linear-NonGaussian	AutoCD	3.0	–	10.0	–	–	–
	GPT-4o-mini	2.0	2.6	10.0	16.8	–	–
	GPT-5.2	2.4	2.6	9.3	15.8	–	–
	Gemini-2.5-Flash	2.5	3.5	8.6	14.7	–	–
	ORCA	2.0	2.2	7.9	12.0	–	–
ER-Nonlinear	AutoCD	3.0	–	10.0	–	39.3	–
	GPT-4o-mini	2.5	2.9	10.1	15.3	41.2	84.6
	GPT-5.2	2.6	3.4	10.2	15.2	40.6	78.1
	Gemini-2.5-Flash	2.3	3.2	10.0	15.5	42.4	80.6
	ORCA	1.4	1.6	10.4	16.3	38.9	76.5
Scale-Free-Nonlinear	AutoCD	2.2	–	4.6	–	–	–
	GPT-4o-mini	2.0	2.4	5.3	6.3	–	–
	GPT-5	1.6	2.4	4.5	6.4	–	–
	Gemini-2.5-Flash	1.6	2.4	5.9	6.7	–	–
	ORCA	0.6	0.9	1.7	3.7	–	–

Table D.3: Causal discovery performance on synthetic datasets at $d = 3$, $d = 5$, and $d = 10$ (mean over 10 runs).

Scenario	Treatment Assignment	Outcome Function	Effect Type	Purpose
Baseline	Linear $P(T W)$	Linear $E[Y T, W]$	ATE	Baseline sanity check
Non-linear Treatment Assignment	Non-linear (e.g., W^2)	Linear $E[Y T, W]$	ATE	Robustness to non-linear assignment
Non-linear Outcome Model	Linear $P(T W)$	Non-linear (e.g., W^2)	ATE	Outcome model misspecification
Heterogeneous Treatment Effects	Linear $P(T W)$	Interaction ($T \times W$)	CATE	Heterogeneous treatment effects

Table D.4: Synthetic data-generating settings for treatment effect estimation experiments.

time limit was exceeded, participants stopped the current task and proceeded to the next session.

- **Phase 3: Post-experiment Survey (10 min)** After completing both conditions, participants answered an overall post-experiment questionnaire including comparative items (ORCA vs. Baseline) and optional open-ended feedback.

E.1.2 Task Pipeline. Both conditions followed the same three-step pipeline and required participants to submit step-wise artifacts:

- (1) **Step 1: Data exploration and dataset retrieval (Data Exploration)** Participants explored the database schema and identified the tables, columns, join paths, and variable definitions required to answer the causal question. **Artifacts:** (i) final SQL query (sql), and (ii) final analysis dataset (data.parquet).

- (2) **Step 2: Causal structure construction (Causal Discovery)** Participants specified or refined a causal graph including the treatment–outcome relation and relevant variables (e.g., confounders, mediators, colliders). **Artifacts:** final causal graph (graph.json).
- (3) **Step 3: Effect estimation and interpretation (Causal Inference)** Participants estimated the average treatment effect (ATE) and provided a brief interpretation, including uncertainty checks when applicable. **Artifacts:** estimation output (ate.json).

E.1.3 Operational Rules and Data Collection. Throughout the study, participants were instructed to externalize their reasoning (think-aloud) by briefly stating what they were attempting and why (or recording short notes). For ORCA, system logs were collected automatically and submitted as a compressed archive. For the Baseline

Synthetic scenario	Method	ATE Abs Error ↓	ATE Bias ↓	CI Coverage (%) ↑	CI Width ↓
Linear Baseline	Causal Forest	0.0283	-0.0283	100.0	0.1620
	DoubleML	0.0113	-0.0067	100.0	0.0585
	T-learner	0.1057	0.1057	—	—
	ORCA (ours)	0.0030	-0.0018	100.0	0.0197
Non-linear Treatment Assignment	Causal Forest	0.0289	-0.0285	100.0	0.1740
	DoubleML	0.0321	-0.0321	40.0	0.0581
	T-learner	0.0643	0.0643	—	—
	ORCA (ours)	0.0053	0.0025	80.0	0.0192
Non-linear Outcome Model	Causal Forest	0.0392	-0.0392	100.0	0.0414
	DoubleML	0.0267	-0.0252	40.0	0.0306
	T-learner	0.0949	0.0949	—	—
	ORCA (ours)	0.0250	-0.0036	100.0	0.0349
Heterogeneous Treatment Effects (CATE)	Causal Forest	0.0125	-0.0044	100.0	0.2123
	DoubleML	0.0157	-0.0013	100.0	0.0746
	T-learner	0.1326	0.1326	—	—
	ORCA (ours)	0.0077	-0.0032	100.0	0.0529

Table D.5: Treatment effect estimation performance on synthetic scenarios (oracle graph setting).

Graph Setting	Method	ATE Abs Error ↓	ATE Bias ↓	CI Coverage (%) ↑	CI Width ↓	Success rate (%) ↑
Oracle Graph	Causal Forest	0.0595	-0.0571	88	1.1089	100
	DoubleML	0.0527	0.0020	84	1.0294	100
	T-learner	0.0270	-0.0071	—	—	24
	ORCA (ours)	0.000032	0.000032	100	1.6338	100
Agentic Graph	ORCA (ours)	0.0414	-0.0407	88	1.0531	100

Table D.6: Treatment effect estimation performance on the REEF benchmark.

condition, the interaction history was recorded via screen capture and/or saved prompts and code, following the same artifact submission structure whenever possible. Participants additionally reported whether they completed each task within the time limit and submitted the final log archive for each run.

E.2 Baseline Implementation

The baseline is implemented as a single, conversation-based LLM assistant with step-gated progression. Within each step, participants interact through free-form dialogue and tool calls for schema inspection, SQL execution, Python-based analysis, and artifact saving. Step transitions are explicitly controlled by a fixed user command (/next), which triggers a finalization routine that requires saving step-specific artifacts (Step1: SQL query and dataset; Step2: causal graph or adjacency representation; Step 3: ATE estimate), as specified in the prompt used for the baseline shown in Figure F.10.

To reflect a basic architecture capable of maintaining continuity across interactions, the baseline is designed to preserve state across steps. Rather than relying on long conversation histories, it maintains a persistent session state (state.json) that records artifact paths and key variables (e.g., treatment, outcome, confounders), and re-injects a compact summary of this state as system context at the start of each step. Step completion is programmatically validated by checking the presence of required artifact types before advance. Overall, this design yields a reproducible, tool-backed workflow

with minimal but explicit state discipline, comparable to common LLM-based coding assistants augmented with execution tools.

E.3 Participants

The study involved $N = 12$ participants with prior experience in data analysis, including undergraduate students (7), master’s students (4), and early-career practitioners (1), drawn from statistics, data science, computer science, and business-related fields. Participants were generally data-literate and reported substantial experience with Python-based analysis, SQL querying, and LLM-assisted coding tools. Figure E.1 provides a detailed characterization of participants’ analytical experience and causal background. As shown in Figure E.1(a), most participants reported intermediate to advanced experience in core analytical skills, reflecting familiarity with common data processing and analysis workflows. Participants also regularly used LLMs to support tasks such as code generation, SQL writing, and exploratory data analysis. While familiarity with high-level causal concepts was widespread (Figure E.1(b)), hands-on experience with formal causal inference methods was less common (Figure E.1(c)). In particular, methods requiring specialized identification strategies such as instrumental variables, difference-in-differences, or regression discontinuity designs were reported by only a small subset of participants.

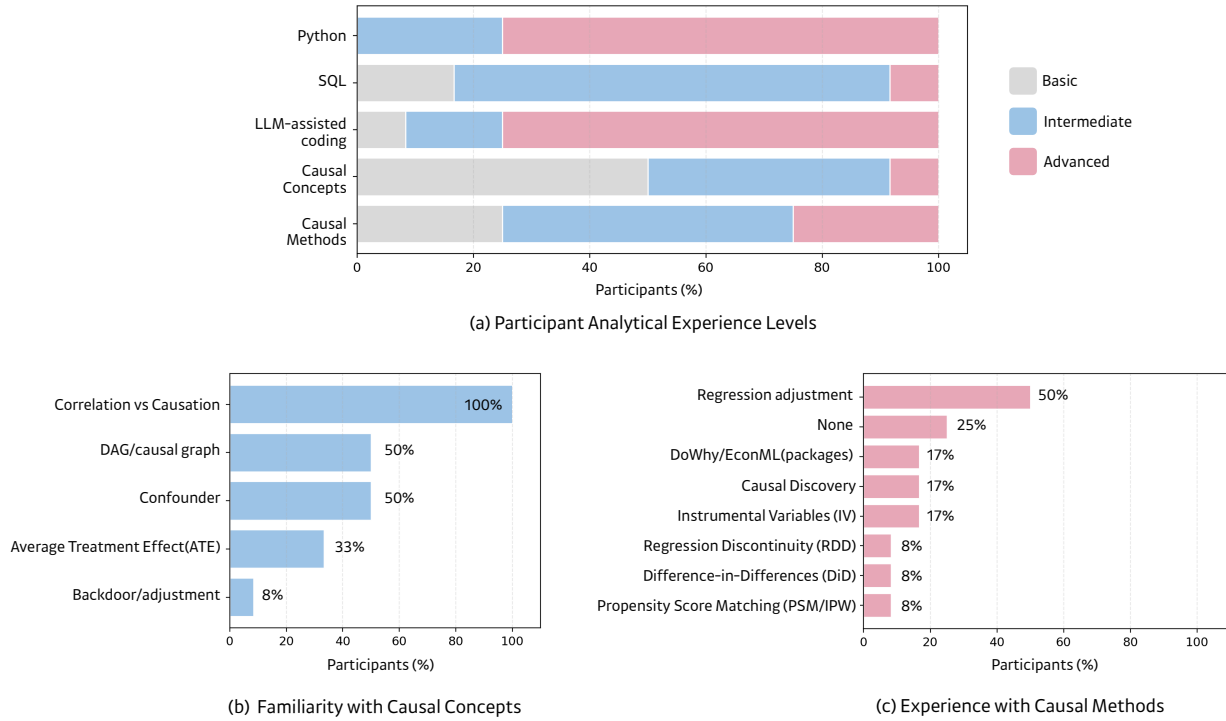


Figure E.1: Overview of participants' analytical background and causal experience ($N = 12$). (a) Distribution of participants' self-reported analytical experience across core skills, including Python, SQL, LLM-assisted coding, and causal analysis, categorized as basic, intermediate, or advanced. (b) Proportion of participants reporting familiarity with causal inference concepts. (c) Proportion of participants reporting prior hands-on experience with specific causal inference methods. Overall, the distributions indicate that participants were generally experienced in data analysis and familiar with core causal concepts, whereas fewer reported direct experience with advanced causal inference methods.

Overall, this participant profile reflects realistic data analysts who routinely work with complex relational data and modern analysis tools, but who often lack extensive training or practical experience in formal causal inference methodologies.

E.4 Quantitative Results

E.4.1 Metric. We report quantitative outcomes for both efficiency and correctness. All metrics were computed per participant, per condition, and per task, and summarized using mean±standard deviation (or median and interquartile range when appropriate).

- **Task Completion:** A run is considered complete if an ATE estimate is successfully produced, indicating that the full analysis pipeline was executed end-to-end, regardless of estimation accuracy.
- **Discovery Success:** A run is marked successful if the inferred causal graph contains both the treatment and outcome nodes.
- **Data Selection Success:** A run is marked successful if the final analysis dataset includes both the treatment and outcome variables.
- **CI Coverage:** Counted when the estimated ATE falls within the confidence interval of the ground-truth effect.

- **ATE Error:** The absolute difference between the estimated ATE and the ground-truth ATE.
- **Normalized Hamming Distance (NHD):** The Hamming distance between the inferred and ground-truth causal graphs, normalized by the maximum possible number of directed edges. Lower values indicate greater structural similarity.
- **User Interaction Time:** The total duration of active user engagement excluding tool execution time. Extreme values are clipped at 0.9 to mitigate the impact of device-level anomalies such as local input/output delays.
- **User Interaction Turns:** The number of additional, non-mandatory user interactions beyond the minimum required workflow, capturing trial-and-error behavior.

E.4.2 Results. Table 1 summarizes user study outcomes across baseline and ORCA conditions. Across all reported metrics, ORCA consistently outperforms the baseline. Specifically, ORCA achieves higher task completion and causal discovery success rates (0.83 vs. 0.75 and 0.83 vs. 0.67, respectively), attains correct dataset for analysis (1.0), and improves confidence interval (CI) coverage (0.50 vs. 0.083), indicating more reliable causal effect estimation outcomes.

Table 2 reports user interaction time and interaction turns across analysis stages. During the data exploration stage, which dominates overall analysis time in relational causal workflows, ORCA significantly reduces user interaction time (80.60s vs. 439.34s on average). Despite this reduction, ORCA maintains comparable or fewer non-mandatory interaction turns across all stages, suggesting that efficiency gains arise from reduced trial-and-error rather than increased user effort.

Metric	β_{cond}	p
Data Exploration Step (s)	-675.49	.029*
Causal Discovery Step (s)	+108.34	.070
Effect Estimation Step (s)	+193.28	.128
Graph Structural Error (NHD)	-0.0079	< .001*
ATE Absolute Error (w/ outliers)	-16.76	.321
ATE Absolute Error (w/o outliers)	-0.0598	.020*

Table E.1: Validation results for each result. β_{cond} represents the estimated difference between ORCA and baseline, controlling for task difficulty and individual differences.

E.4.3 Validation of Results. To rigorously validate our findings under a small-sample ($N=12$) study design, we fitted linear mixed models (LMMs) for each metric with condition (ORCA vs. baseline) and task as fixed effects and participant as a random intercept, controlling for task difficulty and individual differences. Table E.1 reports the estimated condition effects; we use $p = .05$ as the significance threshold.

ORCA significantly reduced data exploration time by an estimated 675 seconds ($p = .029$). The causal discovery and effect estimation steps showed non-significant trends toward modestly increased time ($p = .070$, $p = .128$). This aligns with ORCA’s design intent: by compressing repetitive data exploration, users reallocated effort toward focused causal reasoning. For graph structural error, NHD was lower under ORCA ($p < .001$), confirming improved causal structure learning. For ATE absolute error, the full-data model yielded a non-significant result ($\beta = -16.76$, $p = .321$) due to three participants who misconfigured the analysis dataset and produced errors an order of magnitude larger than the typical range. After excluding these task-configuration errors as outliers ($IQR \times 1.5$), residual variance dropped substantially and the effect became significant under OLS ($\beta = -0.0598$, $p = .020$). We report both results for transparency.

E.5 Survey Questions and Responses

We collected both structured ratings and open-ended responses. The survey included a pre-study questionnaire (background and prior experience), post-task ratings (per condition), and a post-experiment questionnaire comparing ORCA against the Baseline.

E.5.1 Question List.

Pre-study questionnaire (background). Items included: demographics (education level, age range, gender, and field of study), primary data analysis context, Python/SQL experience, debugging familiarity, experience with LLM-assisted coding (e.g., Code Interpreter),

Construct	Baseline	ORCA	Δ
Usability	4.21	4.83	+0.63
Transparency	3.28	5.22	+1.94
Trust	3.33	5.13	+1.79
Mental Effort (R)	3.75	4.00	+0.25
Sense of Agency	3.63	4.58	+0.96

Table E.2: Post-task ratings aggregated by construct (7-point Likert).

and prior causal inference exposure (learning experience, familiarity with concepts, and methods previously used).

Post-task ratings (1–7 Likert). Participants rated the system after each task along the following constructs:

- **Usability / Ease of Use:** (i) “I found this system easy to use.” (ii) “I found it easy to follow the analysis flow.”
- **Transparency:** (i) “I understood why the system reached its conclusion.” (ii) “Intermediate steps (data selection, graph, estimation) were sufficiently explained.” (iii) “It did not feel like the answer was produced suddenly without explanation.”
- **Trust:** (i) “I would use the result of this task for real decision making.” (ii) “When I doubted the result, the system provided clues to verify it.”
- **Workload / Mental Effort (reverse-coded):** “I experienced high mental effort while completing this task.”
- **Sense of Agency:** (i) “I felt that I made the core decisions in the analysis.” (ii) “It felt more like we analyzed together rather than the system analyzing instead of me.”

Open-ended questions (optional). Participants answered: (i) “What was the most confusing moment during this task?” (ii) “Which feature was most helpful during this task?” (iii) Additional comments.

Post-experiment comparative items. Participants responded to comparative statements such as: (i) “ORCA helped me reach a conclusion faster than the Baseline.” (ii) “ORCA reduced trial-and-error (re-asking / re-trying) compared to the Baseline.” (iii) “ORCA’s results felt more trustworthy than the Baseline.” (iv) “I could quickly understand what to do next even as a first-time user.” (v) “The learning cost to become proficient with ORCA was not high.” Participants also selected common blocking points in ORCA (multi-select).

E.5.2 Results.

Post-task ratings. Table E.2 summarizes participants’ post-task ratings on a 7-point Likert scale. Across nearly all dimensions, ORCA received higher average ratings than the Baseline. The largest gains were observed in **Transparency**, where participants reported clearer explanations of intermediate steps and less perception of results appearing as a “sudden output”. Improvements were also observed in **Trust**, with participants indicating greater willingness to rely on results and better availability of clues for verification. Usability and Sense of Agency showed moderate but consistent improvements, while reported mental effort (reverse-coded) was slightly lower for ORCA, suggesting that the added structure did not increase cognitive burden.

Item	Mean	Std.
ORCA helped me reach conclusions faster	4.33	1.97
ORCA reduced trial-and-error	4.67	2.31
ORCA results felt more trustworthy	5.50	1.98
ORCA was easy to orient as a first-time user	4.67	1.56
Learning cost to become proficient was low	4.25	1.54

Table E.3: Post-experiment survey (7-point Likert).

Post-experiment comparative ratings. Table E.3 summarizes participants’ comparative judgments after completing both conditions. Participants generally perceived ORCA as more trustworthy than the Baseline and reported improved clarity in understanding what to do next. Perceived gains in efficiency, such as faster convergence and reduced trial-and-error, were positive on average but showed variability across participants. Overall, these results suggest that ORCA’s primary perceived advantage lies in enhanced clarity and trustworthiness, with efficiency gains depending on individual user experience.

F Agent Prompt Templates

Below we list prompts for each module in ORCA.

Prompt for Table Explorer

You are a professional data analyst specialized in database documentation and exploratory data analysis (EDA).

You will be given a table name and a summary of its columns.

Your task is to provide a structured JSON analysis consisting of the following fields:

1. table_description: (string, 1–2 sentences)
 - Briefly describe the type of data stored, who generates it, and its primary analytical uses.
2. columns: (list of objects)
 - For each column, analytical observations such as:
 1. High null ratio (e.g., "Nulls: 340/1000 (34%)")
 2. Distinct count and duplicates (e.g., "Distinct values: 983/1000 → some duplicates")
 3. Low variance or uniformity (e.g., "All values are 'true'")
 4. Range coverage (e.g., "Date range from 1970-01-10 to 2005-12-01")
 5. Median, min/max (e.g., "Median birth date is 1988-05-18")
 6. Potential outliers or suspicious values, other range issues
 7. Low variance, skewness, duplicates; Uniform or skewed distribution based on median, mean, min/max for numerical columns, and top value counts for categorical columns.

Notes must:

- Provide meaningful insights from a data analysis perspective (e.g., likelihood of being an identifier, presence of outliers, skewed distributions, repeated values, narrow/wide range issues).
- When possible, ****include specific statistics**** (e.g., counts, percentages, ranges, min/max/mean/median).
- Avoid vague language like “high” or “low” without numeric support.
- Be practical advice for downstream analysis, not superficial descriptions
- If multiple notes exist, number them clearly as "1.", "2.", "3.", etc.
- 3. analysis_considerations: (string)
 - Summarize any critical risks, biases, or opportunities associated with this table.
 - Mention how the quality of this table may affect downstream analysis.
 - Mention any preprocessing or transformations that may be needed.

4. recommended_analyses: (list of objects) Considering relationship with other tables, generate up to 3 specific, concrete analysis usecases. Each recommendation should include:

- name: str
- objective: str
- data_requirements: List[str] (table/column names) (e.g., "users.point_balance", "orders.total_price")
- method: str (e.g., cohort analysis, regression, survival analysis)
- assumptions: str
- expected_insights: str

- Output must conform exactly to the JSON structure defined above.

{format_instructions}

=====

[Table Name]

{table_name}

[Column Information]

{column_summary}

[Foreign Key Relationships]

{fk_str}

Figure F.1: Prompt for Table Explorer module. Generates a JSON output including summary statistics, anomalies, and recommended analyses.

Prompt for Table Recommender

You are an experienced and professional database administrator. Your task is to analyze a user question and a database schema to provide relevant information.

The database schema consists of table descriptions, each containing multiple column descriptions. Our goal is to identify the intent of the user and select the relevant tables and columns.

[Instruction]

1. Analyze the user question and decide what information is needed from the database schema. Think step by step.
2. Identify the relevant tables and columns that includes the information needed.
3. Discard any table or columns that is not related to the user question and evidence.
4. Sort the columns in each relevant table in descending order of relevance and keep the top 6 columns.
5. Ensure that at least 3 tables are included in the final output JSON. 6. The output should be in JSON format.

[Requirements]

1. If a table has less than or equal to 5 columns, mark it as "keep_all".
2. If a table is completely irrelevant to the user question and evidence, mark it as "drop_all".
3. Prioritize the columns in each relevant table based on their relevance.

[Example1]

[DB_ID] university_records

[Schema]

Table: student

Table description: Contains information about students enrolled at the university.

[(student_id, unique identifier for each student. Value examples: [101, 102, 103].), (name, full name of the student. Value examples: ['Alice Smith', 'Bob Lee'].), (gender, gender of the student. Value examples: ['M', 'F'].), (birth_date, date of birth. Value examples: ['2000-05-10', '1999-12-31'].), (department_id, department the student belongs to. Value examples: [1, 2, 3].)]

...

[Question] Which student from the Mathematics department had the highest attendance in Calculus during the 2022 Spring semester?

[Answer]

```json

```
{ "student": "keep_all",
 "course": "keep_all",
 "enrollment": ["student_id", "course_id", "attendance", "semester", "grade"],
 "department": "keep_all" }
```

```

Question Solved.

=====

Here is a new example, please start answering:

[DB_ID] {db_id}

[Schema]

{desc_str}

[Foreign keys]

{fk_str}

[Question]

{query}

[Answer]

Figure F.2: Prompt for Table Recommender module. Due to space constraints, only one of the few-shot prompt examples is included, with some parts omitted.

Generation Prompt for Text2SQL Generator

Given a [Database schema] description, a knowledge [Expressions] and the [Question], you need to use valid PostgreSQL and understand the database and knowledge, and then decompose the question into subquestions for text-to-SQL generation.

When generating SQL, we should always consider constraints:

[Constraints]

- In SELECT <column>, just select needed columns in the [Question] without any unnecessary column or value
- In FROM <table> or JOIN <table>, do not include unnecessary table
- If use max or min func, JOIN <table> FIRST, THEN use SELECT MAX(<column>) or SELECT MIN(<column>)
- If [Value examples] of <column> has 'None' or None, use JOIN <table> or WHERE <column> IS NOT NULL is better
- If use ORDER BY <column> ASC/DESC, add GROUP BY <column> before to select distinct values

=====

[Database schema]

Table: patient

[("patient_id", "unique identifier for the patient". Value examples: [1001, 1002, 1003].),
 ("name", "patient name". Value examples: ['Alice', 'Bob', 'Charlie'].),
 ("birth_date", "patient's date of birth". Value examples: ['1985-02-15', '1990-08-23', '1975-12-04'].),
 ("gender", "patient gender". Value examples: ['M', 'F'].),
 ("doctor_id", "doctor in charge". Value examples: [201, 202, 203].)]
 # Table: doctor

...

[Foreign keys]

patient."doctor_id" = doctor."doctor_id"
 appointment."patient_id" = patient."patient_id"
 appointment."doctor_id" = doctor."doctor_id"

[Question] What are the names of patients who had a completed appointment with a cardiologist?

Decompose the question into sub questions, considering [Constraints], and generate the SQL after thinking step by step:

Sub question 1: Which doctors have the specialty 'Cardiology'?

```
```sql
SELECT "doctor_id"
FROM doctor
WHERE "specialty" = 'Cardiology'
```
```

Sub question 2: Which patients had a completed appointment with these doctors?

```
```sql
SELECT DISTINCT T2."name"
FROM appointment AS T1
INNER JOIN patient AS T2
ON T1."patient_id" = T2."patient_id"
WHERE T1."status" = 'completed'
AND T1."doctor_id" IN (SELECT "doctor_id" FROM doctor WHERE "specialty" = 'Cardiology')
```
```

Question Solved

=====

...

[Database schema] {desc_str}

[Foreign keys] {fk_str}

[Question] {query}

Decompose the question into sub questions, considering [Constraints], and generate the SQL after thinking step by step:

Figure F.3: Generation prompt for Text2SQL Generator module. Due to space constraints, only one of the few-shot prompt examples is included, with some parts omitted.

Self-correction Prompt for Text2SQL Generator

[Instruction] When executing SQL below, some errors occurred, please fix up SQL based on query and database info.

Solve the task step by step if you need to.

When you find an answer, verify the answer carefully.

[Constraints]

- In SELECT <column>, just select needed columns in the [Question] without any unnecessary column or value
- In FROM <table> or JOIN <table>, do not include unnecessary table
- If use max or min func, JOIN <table> FIRST, THEN use SELECT MAX(<column>) or SELECT MIN(<column>)
- If [Value examples] of <column> has 'None' or None, use JOIN <table> or WHERE <column> IS NOT NULL is better
- If use ORDER BY <column> ASC/DESC, add GROUP BY <column> before to select distinct values

[Query]

{query}

[Database info]

{desc_str}

[Foreign keys]

{fk_str}

[old SQL]

```sql

{sql}

```

[POSTGRESQL error]

{sql_error}

Now please fix the old SQL and generate new SQL again.

Only output the new SQL in the code block, and indicate script type by ```sql ``` in the code block.

Figure F.4: Self-correction prompt for Text2SQL Generator module, revises faulty SQL codes.

Validation Prompt for Text2SQL Generator

An PostgreSQL code was generated for a user's question. The query executed, and the following results were returned

Your job is to determine:

- If there are results, if the result is sufficient and well-matched to the user's question, answer 'Yes'. if the result is insufficient and didn't answer all user requests, answer 'No' and provide an explanation of why the answer is insufficient.
- If the result is empty, determine if the empty result is from an error in the SQL that, if corrected, would likely return results, or if the SQL logically correct but there was just no matching data. If the SQL logic is sound and the absence of rows is likely due to a lack of matching data, respond only with: 'Yes.' If the SQL has a logical issue (e.g., wrong JOINS, incorrect filters, misused columns) that could explain the lack of results, respond ****only**** with: 'No. <brief explanation>' Note: Do NOT say 'No.' if the SQL logic is sound but the data may be missing. That should be answered with 'Yes.'

Example1:

User Question: "Show all orders from last month."

SQL:

```
```sql
SELECT *
FROM orders
WHERE created_at >= DATE_TRUNC('month', CURRENT_DATE) - INTERVAL '1 month'
AND created_at < DATE_TRUNC('month', CURRENT_DATE);
```
```

Output: Yes. (The SQL logic is correct and filters for the previous month. If there are no matching rows, it's a data issue, not a query issue.)

Example2:

User Question: "Which users purchased groceries this year?"

SQL:

```
```sql
SELECT u.user_id
FROM users u
JOIN orders o ON u.user_id = o.user_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.sku_id = p.product_id
JOIN categories c ON p.category_id = c.category_id
WHERE c.name = 'Groceries' AND EXTRACT(YEAR FROM o.created_at) = EXTRACT(YEAR FROM CURRENT_DATE)
```
```

Output: No. The join condition on oi.sku_id = p.product_id is incorrect. It should likely be oi.sku_id = s.sku_id JOINED to sku and then to products.

Now analyze the following SQL and answer using the same format. Use the schema information provided to determine if the SQL logic is sound.

User Question: {query}

SQL: {sql}

schema info: {desc_str}

Output:

Figure F.5: Validation prompt for Text2SQL Generator module, assessing whether the generated SQL code fulfills the user request and is free of logical errors.

Prompt for Variable Selection

You are a professional data analyst specialized in database documentation and exploratory data analysis (EDA).

You are an expert in causal inference. Your task is to identify the treatment and outcome variables from a user's causal question and available data.

Given:

- A natural language causal question
- A sample of the preprocessed DataFrame
- Table schema information

Identify:

- **Treatment (T)**: The variable that is being manipulated or intervened upon (the "cause")
- **Outcome (Y)**: The variable that is being measured or affected (the "effect")

Rules:

- Treatment should be a variable that can be manipulated or assigned (e.g., "received_treatment", "discount_applied", "campaign_exposed")
- Outcome should be a variable that represents the result or effect (e.g., "purchase_amount", "conversion_rate", "customer_satisfaction")
- Use variable names that exist in the DataFrame columns
- Provide clear reasoning for your selection

{format_instructions}

"""),

("human", ""

User Question:

{question}

DataFrame Sample:

{df_sample}

Table Schema:

{tables}

Available Columns:

{columns}

""")

])

Figure F.6: Prompt for the Variable Selection module of Causal Inference Agent, identifies the treatment and outcome variable based on user query, dataframe, and schema information.

Prompt for Configuration Selection module

You are a causal inference expert using the DoWhy library.

Your job is to choose the appropriate causal inference strategy, estimation method, and optional refutation methods given:

- A user's causal question
- Extracted variables (treatment, outcome, confounders)
 - data type information as `{treatment\_type}` and `{outcome\_type}`
- Basic data preview

Only choose from the valid options defined below.

Causal Tasks:

- `estimating_causal_effect`: estimate average treatment effect (ATE)
- `mediation_analysis`: decompose effect via mediators
- `causal_prediction`: predict outcome using causal structure (e.g., TabPFN)
- `what_if`: simulate counterfactuals
- `root_cause`: identify potential causes of outcome

Identification Strategies:

- `backdoor`: adjust for confounders that affect both treatment and outcome
- `frontdoor`: identify effect using mediators when backdoor not available
- `iv`: use instrumental variables for exogenous variation
- `mediation`: isolate indirect vs direct effects
- `id_algorithm`: automated graphical ID algorithm
- If `outcome_type` is "binary", prefer using `backdoor.generalized_linear_model`
- If `outcome_type` is "continuous", prefer using `backdoor.linear_regression`

Estimation Methods:

- `backdoor.linear_regression`: basic OLS on adjusted data
- `backdoor.propensity_score_matching`: match units by treatment probability
- `backdoor.propensity_score_stratification`: stratify by score and estimate
- `backdoor.propensity_score_weighting`: reweight sample by inverse propensity
- `backdoor.distance_matching`: match using nearest-neighbor distance
- `backdoor.generalized_linear_model`: GLM for non-normal outcomes
- `iv.instrumental_variable`: two-stage least squares using IV
- `iv.regression_discontinuity`: exploit cutoff-based variation
- `frontdoor.two_stage_regression`: mediator-based 2-stage estimator
- `mediation.two_stage_regression`: mediation-specific 2-stage model
- `causal_prediction.tabpfn`: use TabPFN to predict causal outcomes
- `what_if.simple_model`: simulate counterfactual with regression
- `what_if.tabpfn`: simulate counterfactual using TabPFN

Refutation Methods (optional):

- `placebo_treatment_refuter`: randomly replace treatment and re-test
- `random_common_cause`: add synthetic common cause to check stability
- `data_subset_refuter`: re-run analysis on subsets
- `add_unobserved_common_cause`: simulate bias from unobserved variables

{format_instructions}

Figure F.7: Prompt for Configuration Selection module, selects a causal inference strategy including the task, identification, estimation, and refutation based on user query and data context.

Prompt for Interpretation module

You are a causal inference expert. Based on the following information, write a clear but informative explanation of the causal analysis results.

Causal task metadata:

- Task type: {task}
- Estimator used: {estimation_method}
- Estimated causal effect: {causal_effect_value}
- 95% confidence interval: {causal_effect_ci}
- Refutation result (if any): {refutation_result}
- Label mappings (optional): {label_maps}

Parsed query details:

- Treatment variable: {treatment} – {treatment_expression_description}
- Outcome variable: {outcome} – {outcome_expression_description}
- Confounders: {confounders}
- Mediators (if any): {mediators}
- Instrumental variables (if any): {instrumental_variables}

Your goal is to make the result interpretable to a data-literate but non-expert audience. Add an example or intuitive description where possible.

Your explanation should include:

1. A plain interpretation of the estimated causal effect in everyday language.
 - Along with statistical language, provide a translation of directionality (positive/negative effect) into intuitive language (e.g., "users who signed up longer ago are slightly less active").
 2. Whether the effect is statistically significant based on p-value or CI,
 3. Whether the refutation result strengthens or weakens confidence in the finding,
 4. Any caveats or assumptions that should be kept in mind,
 5. If label mappings are provided, interpret the effect in human terms.
- Respond with a concise analytical summary (3–6 sentences).

{format_instructions}

Figure F.8: Prompt for the Interpretation module of Causal Inference Agent, guiding natural-language explanation of causal effect estimates and significance.

Prompt for Grading Criteria

You are a data expert tasked with evaluating the analytical quality of a table description. You will be given a `table\_description` and must assign a score from 0 to 5 based on how well the description:

- Captures outliers
- Analyzes missing values
- Describes distributional characteristics
- Suggests appropriate analysis directions or methods

Go beyond surface-level summaries. Evaluate whether the author shows an understanding of how the data should be interpreted and analyzed, not just what it looks like.

Scoring Rubric (0–5 points)

0 points:

- Purely structural description with no mention of missing values, outliers, or data distributions.
- Only lists column types, basic metadata, or high-level table purpose.

1 point:

- Mentions some missing values or simple statistics (e.g., number of distinct values).
- Lacks any interpretation or analytical context.

2 points:

- Includes null ratios or range for some columns.
- May suggest that missing data or certain value ranges exist, but no specific interpretation or analysis method is provided.

3 points:

- Consistently describes missing values and value distributions across most columns.
- Identifies possible outliers or highly skewed data.
- Mentions basic statistics like mean, median, or top values.
- Provides minor analytical hints but no concrete method suggestions.

4 points:

- Well-rounded analysis of missingness, outliers, and distributions across important columns.
- Offers useful guidance on potential data preprocessing (e.g., log transformation, filtering).
- Considers how data quality or structure affects downstream analysis.

5 points:

- Comprehensive analytical insights, including nulls, outliers, and value distributions.
- Recommends appropriate analysis methods (e.g., regression, classification, time series).
- Discusses assumptions, interpretation directions, or modeling considerations.
- Connects column-level insights to potential analytical goals or questions.

Figure F.9: Evaluation Prompt for Table Description Task. GPT-4o mini use this prompt for scoring table description of table explorer module and baseline(GPT-4o-mini).

Prompt for User Study Baseline

```

## SYSTEM ARCHITECTURE

You work in a **3-STEP SESSION SYSTEM**:

### Step 1: Data Exploration & Dataset Construction
- Understand schema and extract an analysis dataset for the causal question.
- Identify treatment, outcome, and candidate confounders.
- Produce: (a) final SQL query, (b) cleaned dataset.

**Output Artifacts**:
- SQL query (saved as 'sql')
- Clean dataset (saved as 'dataset')

### Step 2: Causal Graph Discovery
- Propose a plausible causal graph over the selected variables (use data + reasoning).
- Optionally use causal discovery tools if appropriate.
- Produce: graph (edge list or adjacency).

**Graph Format**:
```python
graph = {
 "nodes": ["treatment", "outcome", "confounder1", ...],
 "edges": [
 {"from": "confounder1", "to": "treatment"},
 {"from": "treatment", "to": "outcome"},
 ...
]
}

```

```

Output Artifacts:
- Causal graph (saved as 'graph' or 'graph_adj')

Step 3: Causal Effect Estimation
- Estimate ATE using a reasonable identification strategy (e.g., backdoor adjustment).
- Report method, estimate, uncertainty (CI), and adjustment set.
- Produce: ATE result JSON.

Output Format:
```python
ate_result = {
    "treatment": "X",
    "outcome": "Y",
    "method": "backdoor_adjustment",
    "ate": 0.123,
    "confidence_interval": [0.100, 0.146],
    "p_value": 0.001,
    "assumptions": ["no unmeasured confounding", "positivity", ...],
    "adjustment_set": ["Z1", "Z2"]
}

```

```

## TOOL USAGE GUIDELINES

### get_schema(db_id)
### run_sql(db_id, sql)
### run_python(code, context_vars)
### save_artifact(artifact_type, data_ref, step_id, filename)
### final_answer(summary, all_steps_complete)

## WORKFLOW RULES

1. **Step Progression**: ...
2. **Communication**:
    - Be concise but informative
    - Explain your reasoning for methodological choices
    - Ask 1-3 questions to the user when key information is missing
    - Show SQL queries and Python code before execution
3. **Error Handling**:
    - If a query fails, explain the error and suggest fixes
    - If artifact save fails, retry with corrections
    - Validate data quality before proceeding
4. **Artifact Requirements**:
5. **Session Commands**:

```

Figure F.10: Prompt for the user study baseline LLM-assistant with a step-gated workflow for end-to-end causal analysis. The prompt enforces a three-step session structure (data retrieval, causal graph specification, and effect estimation), explicit step transitions, and required artifact saving at each stage, while allowing free-form interaction and tool use within steps.

G Sample Output of modules

This section provides example outputs generated by ORCA modules.

Table Explorer Log

📌 [1] Table Name
`users`

📌 [2] Table Description

The 'users' table stores information about users in the system, including their identification, credentials, personal details, and account status. It serves the purpose of managing user accounts and facilitating user-related operations within the application.

📌 [3] Column Descriptions

- name (character varying, nullable: YES, nulls: 74)
 - 1. High null ratio (7.4%) may indicate incomplete user profiles.
 - 2. The distinct count is 926, suggesting a variety of names.
- email (character varying, nullable: NO, nulls: 0)
 - 1. The distinct count is 983, indicating some repeated email addresses.
- character, nullable: YES, nulls: 340)
 - 1. High null ratio (34%) indicates a significant amount of missing gender data.
 - 2. The distinct count is 2, suggesting this column is likely categorical.

.....

📌 [4] Analysis Considerations

There are significant issues with the data, including a lack of variation in critical fields such as passwords, is_active status, created_at, updated_at, and point_balance. This suggests potential data integrity issues or a lack of proper user account management. Additionally, high null ratios in several fields indicate incomplete user profiles, which could affect user engagement and analysis. The presence of repeated usernames and emails may also pose a risk for duplicate accounts.

📌 [5] Related Tables & Reasons

- `cart`: Linked via `cart.user_id` → `users.user_id`, 1:N, mandatory relationship
- `coupon\_usage`: Linked via `coupon_usage.user_id` → `users.user_id`, 1:N, mandatory relationship

.....

📌 [6] Recommended Analysis

1. User Engagement and Purchase Behavior

- Methodology: Conduct a cohort analysis to segment users based on their engagement metrics (e.g., number of cart additions, coupon usage, and order frequency) over time. Use regression analysis to identify factors that predict higher purchase rates.
- Expected Insights: Identify which user segments are most likely to convert into paying customers and tailor marketing strategies to increase engagement and sales for those segments.

.....

Figure G.1: Execution log of the Table Explorer module, describing a given table by highlighting column specific characteristics, providing relational context and suggesting analysis considerations.

Table Recommender Log

[1] Objective Summary

- Objective: To analyze user information for insights and recommendations.
- Data needed: A table containing user demographics, behavior, and engagement metrics.

[2] Recommended Tables

1. 'users' - user_id, username, name, email, phone_number
2. 'review' - review_id, user_id, product_id, score, created_at
3. 'orders' - order_id, user_id, total_amount, status, created_at
4. 'cart' - cart_id, user_id, product_id, quantity, created_at
5. 'payment' - payment_id, order_id, amount, payment_method, created_at
6. 'point_transaction' - transaction_id, user_id, points_changed, transaction_type, created_at

[3] ERD Image Path

./outputs/images/erd/20250707_174343.png

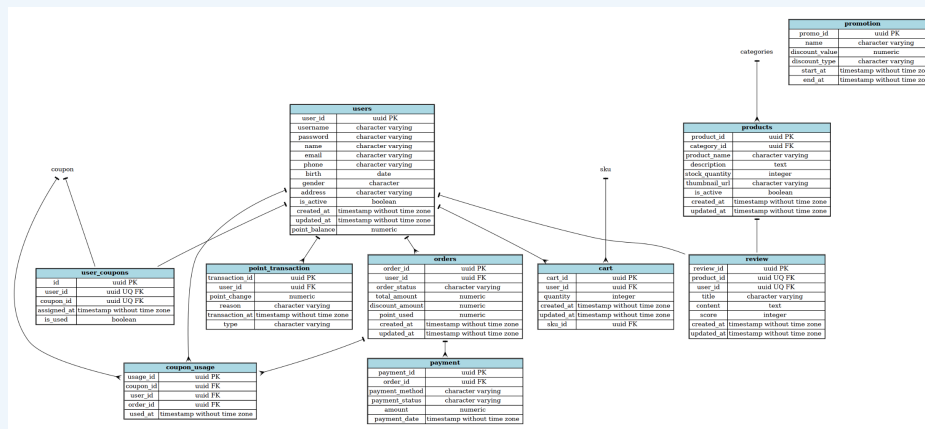


Figure G.2: Execution log of the Table Recommender module, presenting the tables and columns most relevant to the analysis, along with a visualization of their relationships.

Text2SQL Generator Log

 [1] Generated SQL Code

```
SELECT T1.user_id, T1.latest_order_date, COALESCE(T2.total_reviews, 0) AS total_reviews
FROM (
    SELECT user_id, MAX(created_at) AS latest_order_date
    FROM orders
    GROUP BY user_id
) AS T1
LEFT JOIN (
    SELECT user_id, COUNT(review_id) AS total_reviews
    FROM review
    GROUP BY user_id
) AS T2
ON T1.user_id = T2.user_id
```

 [2] SQL Execution Result

user_id	latest_order_date	total_reviews
2a16294d-b477-4936-a58e-c40738490535	2025-07-14 21:21:27.954000	5
1a2f2931-8532-4921-8e8f-71fbd179939e	2025-07-14 21:21:28.595000	2
6f3610f3-a892-47db-a889-91c97a999eb3	2025-07-14 21:21:30.928000	0
8aeea320-f9eb-4744-8885-be6d7a594f20	2025-07-14 21:21:24.087000	2
898da8d0-3be7-42cc-b2fc-eff3f168611b	2025-07-14 21:21:23.783000	3
f50d894f-f02d-4e78-baf9-4feeb4762d34	2025-07-14 21:21:24.530000	5
ee5cb8fc-7b83-496f-b707-649a57a8afd9	2025-07-14 21:21:24.824000	3
c9fac877-76e0-4699-8389-4fb4c8fa2f38	2025-07-14 21:21:30.158000	9
79bf059a-06db-4b1a-b1a3-8db4ba687a44	2025-07-14 21:21:28.454000	7
baf8425a-3dea-4e43-be1a-c15a854c0267	2025-07-14 21:21:30.318000	2

Too many rows returned (1000 rows). Showing top 10.

 [3] Error

No error found. SQL executed successfully!

 [4] LLM Review on Output

 Doublechecked the SQL logic and confirmed it is correct.

Figure G.3: Execution log of the Text2SQL Generator module, displaying the generated SQL, partial results, encountered errors (if any), and module feedback

Causal Discovery Log

Final selected graph:
 Algorithm: PC
 Graph Type: DAG
 Nodes: 5, Edges: 4
 Scores: {'composite_score': 0.99375,
 'markov_consistency': None, 'sampling_stability': 0.9875, 'structural_stability': 1.0}
 Reasoning: Ensemble Synthesis Results:

Top Candidates (ranked by composite score):
 1. PC: composite=0.994, sampling=0.988, structural=1.000
 2. FCI: composite=0.988, sampling=0.988
 3. NOTEARS-nonlinear: composite=0.913, markov_consistency=0.889, sampling=0.858, structural=1.000

Leading Algorithm: PC (composite_score: 0.994)

Data Profile: Pure Continuous, CI reliability: High, Mostly linear: False

PAG Construction:
 - Graph Type: DAG
 - Edge Count: 4
 - Uncertainty Markers: True

DAG Construction:
 - Graph Type: DAG
 - Edge Count: 4

Selection Rationale:
 The ensemble synthesis combines the top 3 candidates using consensus skeleton building and direction resolution. PC was selected as the leading algorithm based on its highest composite score (0.994), which balances markov consistency, sampling stability, and structural stability according to the data characteristics (Pure Continuous, CI reliability: High, Mostly linear: False).

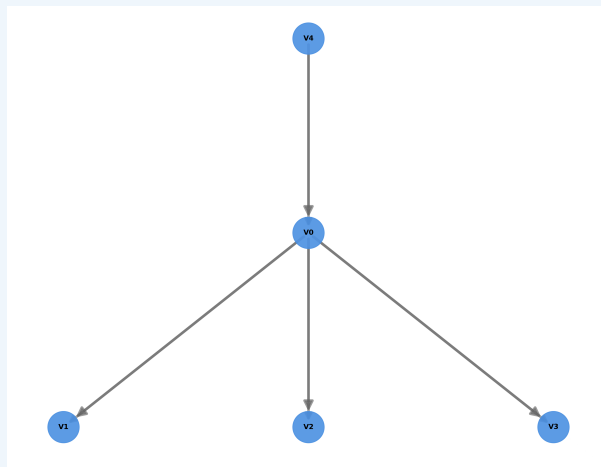


Figure G.4: Execution log of the Causal Discovery Agent. It summarizes candidate graphs produced by multiple discovery algorithms, their composite scores, inferred data characteristics, and the rationale for selecting the final graph. In this example, all algorithms return fully oriented DAGs, so the selected DAG is visualized.

Causal Inference Log

[1] **Parsed Query:** The causal variables extracted from the user query.

```
treatment: signup_days_ago
treatment_expression: DATE_PART('day', CURRENT_DATE - users.created_at)
outcome: is_active
outcome_expression: users.is_active
confounders: []
confounder_expressions: []
mediators: []
instrumental_variables: []
```

[2] **SQL Query:**

```
WITH orders_with_items AS (
  SELECT o.order_id, o.user_id, oi.unit_price, oi.quantity,
         oi.total_price AS order_total, o.total_amount, o.discount_amount
  FROM orders o
  JOIN order_items oi ON o.order_id = oi.order_id
),
...
base AS (
  SELECT
    u.user_id,
    DATE_PART('day', CURRENT_DATE - u.created_at) AS signup_days_ago,
    DATE_PART('year', AGE(u.birth)) AS age,
    u.gender, u.is_active,
    p.order_id, p.unit_price, p.quantity, p.order_total, p.paid_amount,
    c.discount_amount, c.discount_rate,
    COALESCE(uc.used_coupon, FALSE) AS used_coupon,
    (p.paid_amount * 0.01) AS point_earned,
    u.point_balance,
    r.score AS review_score
  FROM payments p
  JOIN users u ON p.user_id = u.user_id
  ...
)

SELECT *
FROM base;
```

[3] **Raw Data Preview:** The first few rows of the extracted raw dataset.

user_id	signup_days_ago	age	gender	is_active	is_active	quantity	order_total	paid_amount
f59d2f4c-c3ee-4c00-...	604	39	F	FALSE	75.93	3	227.79	984.19
F59d2f4c-c3ee-4c00-8574-...	604	39	F	FALSE	378.2	2	756.4	984.19
f59d2f4c-c3ee-4c00-8574-...	604	39	F	FALSE	75.93	3	227.79	984.19
f59d2f4c-c3ee-4c00-8574-...	604	39	F	FALSE	378.2	2	756.4	984.19
77526545-adc2-40be-b451-...	478	24		FALSE	193.81	1	193.81	1288.31

[4] **Strategy:** The causal inference strategy selected by the system.

```
Task: estimating_causal_effect
Identification Method: backdoor
Estimator: backdoor.generalized_linear_model
```

[5] **Final Answer:** The summary of the causal effect.

The analysis indicates that for every day further back a user signed up, their likelihood of being active decreases slightly, with an estimated causal effect of -0.0022 . This means that users who signed up longer ago are just a bit less active than those who signed up more recently. The 95% confidence interval ranges from approximately -0.00225 to -0.00209 , suggesting that this finding is statistically significant and robust, as the interval does not include zero. There were no refutation results, which strengthens our confidence in the finding. It's important to note that this analysis did not control for any confounding variables, meaning the results should be interpreted with caution.

Figure G.5: Execution log of the Causal Inference Agent. It includes the information used through the analysis such as parsed user query, preview of the data, a causal inference strategy, along with an interpretable summary of the estimation result.